AOSP 进阶丛书 · 第一册 · ANDROID 13 (API 33)

Android 13 全栈架构总览

基于 AOSP Android 13 (Tiramisu) 完整源码的全景导读：五层架构、源码树地图、HAL 三代演进、构建与分区体系、ART 运行时、SELinux 与属性系统——每个论断附真实源码路径与行号。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：从应用开发者到系统开发者 3

第一章 五层架构总论 4

第二章 源码树全景导览 6

第三章 内核层与 HAL 层 7

第四章 Native 世界：守护进程与基础库 10

第五章 Framework 层：API、服务与分发中枢 12

第六章 层间通信机制总览 15

第七章 构建与产物体系 17

第八章 运行时体系：ART 与 Zygote 20

第十章 深入属性系统：一次 setprop 的完整旅程 22

第十一章 SELinux：系统的第二套权限体系 25

第十二章 图形与输入骨架概览 26

第十三章 观测与调试：把系统打开给你看 27

第十四章 进阶路线图：本丛书之后怎么走 28

第十五章 设计决策分析（全书总结） 29

第十六章 深水区专题一：SELinux 策略实战与 APEX 内部 30

第十七章 深水区专题二：Soong 实战与链接器命名空间 31

第十八章 边境机制三讲：JNI、Context 家族与 native 智能指针 32

附录 A 版本指纹与阅读路线 34

附录 B 系统进程归属速查 34

附录 C 常用系统属性速查（带源码出处） 35

附录 D 术语表 35

附录 E 上手实验（半天量） 36

第十九章 系统服务注册全流程走读：一个服务的五跳人生 36

第二十章 一次点击的全程旅行：从指尖到像素的九跳（跨册整合） 41

第二十一章 进程优先级总览：从 oom_adj 到 cgroup 冻结 45

教科书习题集（分级） 48

术语表（按拼音排序） 50

排障手册（按"我的问题是..."索引） 52

本册速览

五层架构与贯穿机制（Binder/属性/SELinux/APEX）——任何模块 30 秒定位
源码树地图 · 分区与构建（Soong/APEX/AVB） · HAL 三代演进 · ART 三级执行
深水区：SELinux 策略实战 · apexd 回滚 · Soong 写模块 · 链接器命名空间
边境三讲：JNI 三类引用 · Context 家族 · sp/wp/RefBase · Handler/Looper
建议顺序：通读建框架 → 对照锚点精读 → 附录 E 实验 → 进第二册

前言：从应用开发者到系统开发者

本书是《AOSP 进阶丛书》的第一册，基于你本机的 **AOSP Android 13（Tiramisu，API 33，安全补丁 2022-08-05）** 完整源码树撰写，所有结论都给出了 **文件:行号** 级别的源码锚点，可在 `/run/media/marcusgu/SoftWare/aosp` 下逐一验证。

作为应用开发者，你已经熟悉 SDK 的世界：Activity、Service、Handler、Gradle。而系统开发者的世界从这里开始向下延伸——你的代码之下的 `framework.jar` 是怎么运行起来的，`getSystemService` 拿到的到底是什么，开机 logo 之后到 Launcher 出现之前发生了什么。本丛书沿着“**开机 → 系统就绪 → 应用启动 → 进程间通信**”这条贯穿全栈的主线组织内容：

- **第一册（本书）**：全景地图。五层架构、源码树导览、构建与分区体系、运行时体系。读完你将获得“任何模块都能定位到它属于哪一层、住在哪个目录、依赖谁”的能力。
- **第二册**：系统启动主线。init → Zygote → SystemServer 的逐函数精读。
- **第三册**：Binder IPC 全链路。从内核驱动到 Java 层的完整调用链剖析。

阅读方法建议：先通读建立框架，再对照源码锚点精读；每册末尾的“设计决策”一章是面试与架构能力的分水岭，值得反复咀嚼。

先修要求与学习路线

先修：熟悉 Java/Kotlin Android 应用开发（Activity 生命周期、Service、Handler 基本用法）。不需要 C++ 或 Linux 内核基础。

读完本书你将能够：

- 在 30 秒内定位任何 AOSP 模块属于哪一层、住在哪个目录、依赖谁
- 画出 Android 五层架构图并解释每层的职责与边界
- 理解 Binder/属性/SELinux/APEX 四条贯穿机制各自解决什么问题
- 读懂 Android.bp 构建文件并理解 Soong 构建管线
- 独立完成附录 E 的五个动手实验
- 为后续第二册（启动主线）和第三册（Binder）建立完整的知识地图

本书不覆盖（后续册深入）：进程调度细节（书4）、窗口管理（书5）、图形渲染管线（书8）、Binder 协议逐跳分析（书3）。

> **设计洞察 · 本丛书的源码约定**
>
> 本书所有行号基于 Android 13（TP1A）源码树实测；目录路径均相对源码树根 `$A`。源码树严格只读——研读系统代码的首要纪律是：先理解，再动手。

第一章 五层架构总论

设计洞察 · 学习目标

1. 画出五层架构图；2. 解释每层核心职责与依赖方向；3. 说出三条贯穿机制；4. 追踪一行代码从App到内核

Android 的软件栈通常被划分为五层。分层的意义不在于“好看”，而在于**依赖方向的确定性**：每一层只依赖它下面的一层，越往上越贴近应用，越往下越贴近硬件。

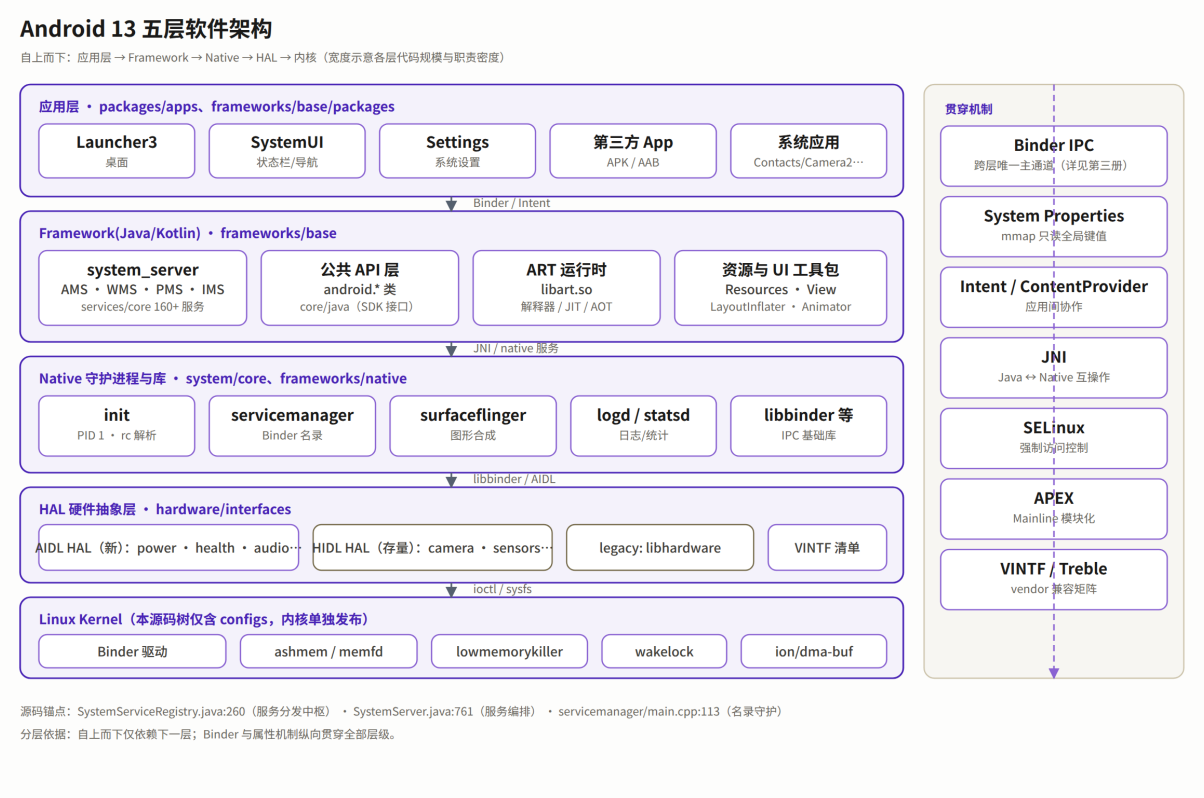


图 1 Android 13 五层软件架构总图：左侧自上而下为应用层、Framework、Native、HAL、内核，右侧为纵向贯穿的公共机制

1.1 自上而下看五层

应用层运行在独立的 App 进程中，通过 SDK API 与系统交互。系统应用大多住在源码树两处：`packages/apps/`（Launcher3、Settings、Camera2 等）与 `frameworks/base/packages/SystemUI`。它们与第三方 App 本质相同——都靠 Binder 调用系统服务，只是签名与权限不同。

Framework 层是系统开发者的主战场，物理上由 `frameworks/base` 承载。它包含两部分：公共 API（`core/java/android/`，即编译进 `framework.jar` 的 `android.*` 类）与系统服务实现（`services/core/java/com/android/server/`，编译进 `services.jar`，运行在 system_server 进程）。你调用的每个系统 API，最终几乎都穿过前者、经 Binder 落到后者。

Native 层是 C/C++ 的世界：`system/core` 与 `frameworks/native` 中的守护进程（init、servicemanager、surfaceflinger、logd 等）和基础库（libbinder、libutils、libcutils）。渲染合成、输入分发这类对性能极敏感的职责都在这一层完成。

HAL（硬件抽象层）把“芯片相关”与“系统无关”隔离开：接口定义集中在 `hardware/interfaces/`，实现由各 SoC 厂商提供在 vendor 分区。Android 13 的主线是 AIDL HAL，存量 HIDL 仍大量并存。

内核层是 Linux kernel 加上 Android 特有补丁（Binder 驱动、ashmem/memfd、lowmemorykiller、wakelock 等）。值得注意的是：**本源码树的 `kernel/` 目录只有 configs、prebuilts、tests 三个子目录，不含内核源码**——AOSP 的内核是独立仓库（如 android13-5.10），但内核的稳定 UAPI 头（包括 Binder 协议的 `binder.h`）通过 `external/kernel-headers/original/uapi/` 进入编译。

1.2 右侧“贯穿列”：真正的系统观

初学者容易把五层理解成“五块积木”，但真正的架构能力在于看见**纵向贯穿的机制**：

- **Binder IPC** 几乎是所有跨层调用的唯一通道：App → system_server 是 Binder，system_server → HAL 服务还是 Binder。第三册将逐跳剖析它。
- **System Properties** 是全机共享的键值区：init 建立，所有进程 mmap 只读共享，`persist.` 前缀落盘。它是比 Binder 轻得多的“单向广播”。
- **SELinux** 的策略覆盖从 init 第一个动作到每个服务的每一次访问——`system/sepolicy/` 是策略源码，理解它才能理解“为什么这个服务连不上那个端口”。

1.3 实战：一行代码看穿五层

以应用里最常见的调用为例——`powerManager.isInteractive()`（判断屏幕是否点亮）：

java 应用进程内（五层穿越起点）

```
PowerManager pm = context.getSystemService(PowerManager.class);
boolean lit = pm.isInteractive();
```

它的下行路径是：`PowerManager`（应用层 API，`core/java/android/os/PowerManager.java`）持有一个 `IPowerManager` Binder 代理 → `SystemServiceRegistry` 在首次获取时经 `ServiceManager.getService("power")` 从 **servicemanager**（Native 层）查到 system_server 注册的电源服务 Binder 句柄 → 调用发起后经 **Binder 驱动**（内核层）进入 system_server（Framework 服务层）的 `PowerManagerService.isInteractive()`（`services/core/.../power/PowerManagerService.java`）→ 返回值原路回到应用。一行代码，五层全穿。**本丛书剩余两册，就是把这条线上的每一层拆开讲透。**

> **设计洞察 · 分层的本质是依赖管理**
> 五层中任何一层被替换，只要接口契约不变，上下层不受影响。这正是 Treble 工程的落点：把 vendor 相关的 HAL/内核钉死接口，让 system 分区可以独立升级。判断一个改动“该放在哪一层”的系统思维：能否被多个应用复用 → Framework；是否性能关键且稳定 → Native；是否因硬件而异 → HAL。

第二章 源码树全景导览

> **设计洞察 · 学习目标**
> 1. 说出五个核心目录；2. 30秒定位任意函数；3. 判断新模块该放哪层

面对数 GB 的源码树，系统开发者的第一项能力是“看目录知职责”。本章给出一张地图，并标出本丛书的精读路径。



图 2 AOSP 源码树地图：上半部分为核心目录分组，下半部分为“从开机到 App”主线的落点

2.1 必须烂熟于心的五个目录

目录	职责	关键子目录
`frameworks/base`	Framework 全部	core/java（API）、services（服务端）、cmds/app_process（Zygote）
`system/core`	基础组件	init、rootdir（init.rc）、debuggerd、libutils

目录	职责	关键子目录
frameworks/native	图形与 IPC	libs/binder（libbinder）、cmds/servicemanager、services/surfaceflinger
hardware/interfaces	HAL 接口定义	power、audio、health 等 AIDL/HIDL 接口与 VINTF 矩阵
art	运行时	runtime（虚拟机）、dex2oat（AOT 编译器）、compiler

其余目录按需检索：packages/（系统应用）、bionic（Android 版 libc）、libcore（Java 核心库，含 OpenJDK 移植 ojluni）、build/（构建系统）、external/（数百个第三方项目）、prebuilts/（预置工具链）。

2.2 用目录回答三个问题

“servicemanager 在哪？” 它在 frameworks/native/cmds/servicemanager/（本树位置；较老教材写 system/core/servicemanager，Android 13 已迁移）。这提醒我们：**教材会过时，目录检索能力不会**。

“init.rc 有几份？” 入口是 system/core/rootdir/init.rc，它 import 了 init.${ro.hardware}.rc、init.${ro.zygote}.rc 等；每个守护进程还各自带 /system/etc/init/*.rc。第二册将精读这条 import 链。

“我该改哪层的代码？” 先看现象的层级：API 行为问题查 frameworks/base/core/java 与 services/core；性能/图形查 frameworks/native；设备差异查 hardware/interfaces 与 device/。

> **源码锚点 · 快速检索工具链**
> ripgrep 全树检索：rg -t cpp "SecondStageMain" --line-number
按文件名定位：find . -name "ZygoteInit.java" -not -path "*/out/*"
确认版本：build/make/core/version_defaults.mk（SDK_INT 33）
排除产物：始终加 -not -path "*/out/*" 避免命中构建产物

第三章 内核层与 HAL 层

> **设计洞察 · 学习目标**
> 1. 列举五项内核增强；2. 解释HAL三代演进；3. 读懂HAL目录结构

3.1 Android 对 Linux 内核的增强

Android 内核 = 标准 Linux + 一组"Android 通用内核"补丁。最重要的五项：

- **Binder 驻动**（`drivers/android/binder.c`，不在本树）：进程间通信的基石，一次拷贝与内核级身份校验都由它实现（第三册详述）。UAPI 定义在 `external/kernel-headers/original/uapi/linux/android/binder.h`。
- **ashmem/memfd**：匿名共享内存，Binder 传递大块数据（如位图）的配套机制。
- **lowmemorykiller 演进**：内核态 LMK 已被用户态 `lmkd`（`system/core/lmkd`）+ PSI 压力信息取代，通过 socket 与 init 体系协同。
- **wakelock/wakeup source**：电源管理的同步原语，`libsuspend`（`system/core/libsuspend`）封装。
- **ION/DMA-BUF**：显存缓冲分配，现已被内核标准 DMA-BUF 取代。

3.2 HAL：三代演进

HAL 的历史是 Android 对“碎片化”宣战的历史。legacy HAL 把厂商代码编进 Framework 进程，升级系统必须重编厂商代码；Android 8 的 Treble 用 HIDL 强制进程隔离与版本化；Android 10 起 AIDL HAL 统一语言栈，让 HAL 与系统服务共用同一套 Binder 设施与工具链。



图 3 HAL 三代演进时间轴与 AIDL HAL 调用路径、VINTF 兼容矩阵

3.4 完整走读：以 Power HAL 为例看穿一个 AIDL HAL 的全部文件

以 `hardware/interfaces/power/aidl/` 为例——这是本树中最简洁的 AIDL HAL 参考实现：

文件一：接口定义（.aidl）

java hardware/interfaces/power/aidl/android/hardware/power/IPower.aidl

```
// 讲解注释①：接口文件是 HAL 的"合同"——
// framework 和 vendor 都依据它生成绑定代码
// :24
interface IPower {
// 讲解注释②：每个方法就是一条"合同条款"
void setMode(in Mode type, in boolean enabled);
```

```
bool isModeSupported(in Mode type);
int getHintSessionPreferredRate();
}
// 辅助类型也在同目录：
// Mode.aidl（枚举：INTERACTIVE/DOUBLE_TAP_TO_WAKE...）
// Boost.aidl（枚举：DISPLAY_UPDATE_IMMINENT...）
// WorkDuration.aidl（工作负载时长报告）
```

文件二：参考实现（default/Power.cpp）

cpp hardware/interfaces/power/aidl/default/Power.cpp

```
// 讲解注释③：厂商实现继承生成的 Binder 接口
// ndk::ScopedAStatus 是 AIDL C++ 的返回值包装
using ::aidl::android::hardware::power::IPower;
class Power : public BnPower {
ndk::ScopedAStatus setMode(Mode type, bool enabled) override {
// 讲解注释④：真正写硬件的代码在这里
// 例如：echo 1 > /sys/power/... 或 ioctl(...)
return ndk::ScopedAStatus::ok();
}
};
```

文件三：构建描述（default/Android.bp）

text default/Android.bp（五要素模板）

```
aidl_interface {
name: "android.hardware.power", // 模块名
vendor_available: true, // vendor 分区可用
srcs: ["android/hardware/power/*.aidl"],
}
cc_binary {
name: "android.hardware.power-service.example",
relative_install_path: "hw", // /vendor/bin/hw/
init_rc: [":android.hardware.power.rc"], // init 启动脚本
vintf_fragments: [":android.hardware.power.xml"], // VINTF 清单
vendor: true, // 归入 vendor 分区
srcs: ["Power.cpp", "main.cpp"],
shared_libs: ["libbinder_ndk", "liblog"],
}
```

文件四：init 脚本（.rc）

text .rc 文件（init 从这里启动 HAL 服务）

```
service vendor.power-hal /vendor/bin/hw/android.hardware.power-service.example
class hal // init 的 hal 类启动（先于 framework 服务）
user system
group system
seclabel u:r:hal_power:s0 // SELinux 域标签
```

文件五：VINTF 清单（.xml）

xml VINTF 清单（告诉 framework "这个设备支持什么 HAL"）

```
<hal format="aidl">
```

```
<name>android.hardware.power</name>
<fqname>IPower/default</fqname> // 实例名
</hal>
// framework 的 compatibility_matrices 里声明需要什么——
// 构建期对齐两份清单，不匹配 = 拒绝构建
```

文件六：SELinux 三件套

text SELinux（vendor/sepolicy/）

```
hal_power.te: // 域定义（allow 规则）
type hal_power, domain;
allow hal_power power_service:service_manager find;
file_contexts: // 文件标签
/vendor/bin/hw/android.hardware.power-service.example
u:object_r:hal_power_exec:s0
service_contexts: // 服务标签
android.hardware.power.IPower/default
u:object_r:hal_power_service:s0
```

设计洞察 · 逐段精读：为什么要六个文件

这是 Treble 的"合同精神"：接口（.aidl）是合同条款、实现（.cpp）是履约、构建（.bp）是打包、启动（.rc）是交付、VINTF 是公证、SELinux 是守卫。**少一个文件，这个 HAL 就无法被 framework 正确发现和使用**。这就是为什么书14 说"五步走一个 HAL"——每步对应这里的文件。

设计洞察 · 为什么 HIDL 被 AIDL 取代

HIDL 达成了隔离目标，但代价是第二套 IDL 编译器、第二个 binder 域（/dev/hwbinder）、第二个服务管理器（hwservicemanager）。AIDL HAL 复用主线设施后，工具链、调试手段、线程模型全部归一——工程上“少一套系统”的收益压倒了迁移成本。存量 HIDL（如部分 camera 接口）长期冻结维护，读代码时先看 `aidl/` 还是 `1.x/` 目录区分代际。

第四章 Native 世界：守护进程与基础库

设计洞察 · 学习目标

1. 说出核心守护进程名录及各自职责；2. 解释 class core/main/hal 的调度含义；3. 定位 libbinder/libutils/libcutils/libgui 四大基础库；4. 用 bootchart 观测启动阶段耗时

Native 层的守护进程几乎全部由 init 在启动各阶段拉起（谁在什么阶段启动，第二册的触发链图给出完整答案）。本册先建立名录。

4.1 核心守护进程名录

进程	源码位置	职责	启动类别
init	system/core/init	PID 1，解析 rc、孵化与监护所有服务	内核第一个用户态进程
servicemanager	frameworks/native/cmds/servicemanager	Binder 服务名录（句柄 0）	class core
surfaceflinger	frameworks/native/services/surfaceflinger	图形图层合成、VSync 分发	class core
logd	system/logging/logd	全机日志汇聚	class core
lmkd	system/memory/lmkd	低内存查杀	class core
vold	system/vold	卷管理、加密、fuse	early-fs 阶段
netd	system/netd	网络策略与 DNS	class main
audioserver / cameraserver / mediaserver	frameworks/av 等	媒体相关服务进程	class main
bootanimation	frameworks/base/cmds/bootanimation	开机动画（开机完成的视觉信号）	class core animation

“类别”（class core/main/hal…）不是注释，而是 init 批量启动的调度单位——`class_start main` 一条命令拉起整组服务，详见第二册 `builtins.cpp` 的 `do_class_start`。

4.2 基础库版图

- **libbinder**（`frameworks/native/libs/binder`）：Native 侧 Binder 运行库，ProcessState/IPCThreadState 所在地，第三册主角。
- **libutils**（`system/core/libutils`）：RefBase、sp/wp 智能指针、Looper、Thread——Native 世界的“公共底座”。
- **libcutils**（`system/core/libcutils`）：属性读写、socket 工具、trace 等 C 工具集。
- **libgui / libui**（`frameworks/native/libs/{gui,ui}`）：BufferQueue、Surface、图形缓冲——渲染栈的 Native 骨架。

> **源码锚点 · 从守护进程到“服务类”的对应**
> servicemanager.rc：class core animation，critical —— 名录自身也是被 init 管理的服务
init.zygote64.rc：class main —— Framework 全部进程的祖先进“主类”
SystemServer.java:435：sMaxBinderThreads=31 —— system_server 的 Binder 线程预算

4.3 启动性能观测的一件老兵器：bootchart

`system/core/init/bootchart.cpp` 是 init 内建的启动时间线采集器：设置 `persist.init.bootchart` 后，init 记录各 action/服务时间戳，配合

`system/core/init/grabbootchart.sh` 产出甘特图。它是第二册“触发链”一章的天然实验伙伴——改一个 rc 触发顺序，bootchart 立刻告诉你启动时间花在哪。Google 现代工作流更多用 perfetto（`external/perfetto`），但 bootchart 与 init 代码同源同寿命，读懂它等于复习 init 的数据结构。

第五章 Framework 层：API、服务与分发中枢

> **设计洞察 · 学习目标**
> 1. 区分 framework.jar 与 services.jar 的部署位置与加载进程；2. 走通 getSystemService 的静态获取链；3. 背出常用系统服务速查表的关键行；4. 解释服务集中 system_server 与独立进程的划分依据

5.1 一份代码，两个 jar

Framework 的物理形态是两个部署单元：`framework.jar`（所有进程共享的 API 与基础类）与 `services.jar`（只在 system_server 进程加载的服务实现）。App 进程通过 Zygote 的写时复制共享 `framework.jar` 的内存映射——这是全机内存优化的大头（第二册 Zygote 章详述）。

API 与实现的边界藏在 `frameworks/base` 的目录结构里：`core/java/android/app/ActivityManager.java` 是给应用调用的门面，真正逻辑在 `services/core/java/com/android/server/am/ActivityManagerService.java`（1.8 万行）。门面通过 Binder 把请求送进 system_server。

5.2 getSystemService 的分发中枢

应用里再平常不过的一行 `context.getSystemService(Context.POWER_SERVICE)`，背后是 `frameworks/base/core/java/android/app/SystemServiceRegistry.java` 这个中枢：

> **源码锚点 · SystemServiceRegistry：服务注册表**
> SystemServiceRegistry.java:260 类定义 —— 静态注册 100+ 服务的 fetcher
SystemServiceRegistry.java:270 SYSTEM_SERVICE_FETCHERS 全局 map
SystemServiceRegistry.java:284 CachedServiceFetcher：进程级缓存的管理器对象
SystemServiceRegistry.java:289+ 各服务以 registerServiceFetcher 注册获取逻辑

多数 fetcher 的逻辑是：拿 Context → 通过 `ServiceManager.getService("power")` 拿到 Binder 代理 → `IPowerManager.Stub.asInterface` 包装 → 缓存。也就是说，**管理器对象只是 Binder 代理的本地缓存马甲**。看懂这一点，你就打通了“应用层 API → Binder → system_server”的静态地图；动态的每一步在第三册。

5.3 system_server 里的服务版图

system_server 以单进程容纳 160+ 系统服务（第二册给出启动时序），按包名归类：`am/`（AMS 及进程/广播/服务管理）、`wm/`（WMS 与窗口/ATM）、`pm/`（PMS 包管理）、`power/`、`input/`、`display/`、`notification/`、`usb/`……而图形合成（SurfaceFlinger）、音频（audioserver）等因性能与崩溃隔离**有意留在外面**——集中与隔离的权衡，是第十章设计决策的主题之一。

5.4 常见系统服务速查表

getSystemService 返回	Binder 名	实现类（services/core/...）	所在进程
ActivityManager	activity	am/ActivityManagerService	system_server
WindowManager	window	wm/WindowManagerService	system_server
PackageManager	package	pm/PackageManagerService	system_server
PowerManager	power	power/PowerManagerService	system_server
InputManager	input	input/InputManagerService	system_server
DisplayManager	display	display/DisplayManagerService	system_server
NotificationManager	notification	notification/NotificationManagerService	system_server
LocationManager	location	location/LocationManagerService	system_server
AudioManager	audio	（跨进程）audio 服务	audioserver
CameraManager	camera	（跨进程）camera 服务	cameraserver
SensorManager	sensors	（跨进程）sensorservice	sensorserver
WifiManager	wifiservice	wifi/WifiService	system_server

注意“所在进程”一列：多数框架服务集中在 system_server，但音频/相机/传感器有意独立进程——再次呼应“故障域”划分。第三册会以 `activity` 服务为例，把“Binder 名 → 代理对象 → 具体方法”的运行期解析全程展开。

5.5 system_server 服务全表（按启动批次分组）

这是全书信息密度最高的一张表——**每行一个服务，一眼看到 system_server 的全貌**。

Bootstrap 批（依赖地基，:1052-1330）

序	服务类名	Binder 名	一句话职责
1	Watchdog	—	死锁看门狗
2	PlatformCompat	—	平台兼容性框架
3	FileIntegrityService	—	fs-verity 文件完整性

序	服务类名	Binder 名	一句话职责
4	**Installer**	installer	installd 客户端（dexopt）
5	DeviceIdentifiersPolicyService	—	设备标识合规
6	UriGrantsManagerService	uri	URI 授权
7	PowerStatsService	—	能耗统计
8	**ATM + AMS**	activity	调度中枢（书4）
9	DataLoaderManagerService	—	增量安装
10	IncrementalService	—	增量数据加载
11	**PowerManagerService**	power	电源策略（书9）
12	ThermalManagerService	—	热管理（书9）
13	HintManagerService	—	性能暗示
—	boot phase 100	—	等默认屏
14	**PackageManagerService**	package	包管理（书6）
15	UserManagerService	user	多用户
16	OtaDexoptService	—	OTA 后 dex 整理

Core 批（轻量统计，:1336）

服务	Binder 名	职责
SystemConfigService	—	系统配置
BatteryService	batterymanager	电池状态
UsageStatsService	usagestats	使用统计
WebViewUpdateService	webviewupdate	WebView 版本
CachedDeviceStateService	—	设备状态缓存
BinderCallsStatsService	—	Binder 调用统计
RollbackManagerService	—	升级回滚

Other 批（大合唱 224 段，:1404-3059）

关键服务	Binder 名	职责	所在册
InputManagerService	input	输入分发（书5）	
WindowManagerService	window	窗口管理（书5）	
NetworkManagementService	network_management	网络管理（书10）	

关键服务	Binder 名	职责	所在册
ConnectivityService	connectivity	连接管理（书10）	
NotificationManagerService	notification	通知（书13）	
LocationManagerService	location	定位	
AudioService	audio	音频策略（书12）	
MediaSessionService	media_session	媒体会话	
BluetoothService	bluetooth_manager	蓝牙	
WifiService	wifi	Wi-Fi（书10）	
TelephonyRegistry	telephony.registry	电话状态（书19）	
MountService（vold 客户端）	mount	存储（书11）	
JobSchedulerService	jobscheduler	后台调度（书9）	
AlarmManagerService	alarm	闹钟（书9）	
DeviceIdleController	deviceidle	Doze（书9）	
DropBoxManagerService	dropbox	系统日志收集	
…（200+ 服务）	…	…	…

设计洞察 · 逐段精读：为什么这张表重要

面试时"说出 system_server 里 10 个服务及职责"是高频题——这张表就是答案。更重要的是**理解启动顺序=依赖顺序**：Installer 先于 AMS（AMS 装系统包要 dexopt）、PMS 先于 WMS（窗口要查包）。顺序错一个=启动失败。

第六章 层间通信机制总览

设计洞察 · 学习目标

1. 对比四种机制；2. 追踪属性set全链；3. 解释不对称设计

系统的“血管系统”由四条通道构成，各自解决不同问题：

机制	典型用途	数据形态	成本量级
Binder IPC	一切跨进程方法调用	结构化事务（Parcel）	微秒级，一次拷贝
System Properties	全局状态广播（开机完成、USB 配置）	键值字符串（mmap 共享读）	纳秒级读
Intent/ContentProvider	应用间协作与数据共享	语义化消息/表格数据	Binder 之上封装

机制	典型用途	数据形态	成本量级
JNI	Java ↔ Native 边界	函数调用	纳秒级（经 ART 精心优化）

属性系统值得单说：init 在第二阶段调用 `PropertyInit()`（`init.cpp:887`）建立 `/dev/__properties__` 共享区，所有进程从此 **mmap 只读**读取；写入则经 socket 连到 init 的属性服务（`property_service.cpp:527` 的 `handle_property_set_fd`），由 init 统一裁决（含 SELinux 检查）后广播。**读路径零系统调用、写路径集中仲裁**——这个不对称设计以极低成本支撑了全机数千项配置。

6.1 一次 `SystemProperties.set` 的完整下行链

把上一节的机制落到代码，从应用层下行到 init 的每一跳：

text 属性写入全链（自上而下）

```
应用：SystemProperties.set("debug.hwui.renderer", "skiagl")
↓ (frameworks/base/core/java/android/os/SystemProperties.java, native_set)
JNI：android_os_SystemProperties_set (core/jni/android_os_SystemProperties.cpp)
↓ __system_property_set
bionic：发送 socket 到 /dev/socket/property_service (libc/bionic/system_properties.cpp)
↓ init 侧 accept4
init：handle_property_set_fd (property_service.cpp:527)
↓ 凭证 SO_PEERCRED + SELinux 上下文
仲裁：HandlePropertySet(:481) → PropertySet(:176) → 更新共享区 + 持久化(persist.)
↓ 属性变更通知
消费方：poll 属性 fd 或由 init 触发 on property:xxx=* 动作
```

对照第三册即将展开的 Binder 链，你会发现惊人相似的结构：**Java 门面 → JNI → Native 库 → 内核/守护进程仲裁 → 广播反馈**。Android 的“跨边界调用”几乎都是这个形状，差别只在仲裁者是谁（init 属性服务、Binder 驱动或 servicemanager）。

6.2 运行时资源覆盖（RRO）：不改代码的定制面

分区里的 overlay APK（书6 §9 扫描的 :320）在运行时由 **OverlayManagerService**（`services/core/.../om/OverlayManagerService.java`）挂载：把覆盖包的资源表叠加进目标包的 AssetManager——**按资源 ID 重定向**，同一 ID 优先取覆盖值。三层用途递进：厂商暗色主题/默认值定制（vendor overlay）、运行时主题切换（framework 的 android 覆盖）、用户个性化（ThemePicker）。它和 APEX 是一对：APEX 换"代码与库"，RRO 换"资源与配置"——都是"不动 system 分区本体"的定制哲学。读码入口：OMS 的 `setOverlayEnabled` 与 `OverlayActorEnumerator` 策略校验（谁能覆盖谁，受 targetName/签名约束）。

> **设计洞察 · 为什么属性不用 Binder**
>
> 属性读取发生在进程启动最早期（连 ART 都未就绪），也发生在 SELinux 策略加载前，必须是一个“dumb 到不能再 dumb”的机制。mmap 只读 + init 单点写，正是不依赖任何运行时的答案。

第七章 构建与产物体系

设计洞察 · 学习目标

1. 画出boot.img组成；2. 理解Soong/Kati角色；3. 列出五大分区

7.1 分区布局



图 4 Android 设备分区布局：引导链、四大主分区与 data 系分区，附五大设计动机

分区的关键词是**解耦与可替换**：system 可刷 GSI 通用镜像，vendor 受 VINTF 接口矩阵约束独立升级，product 承载机型差异。Android 13 还引入 init_boot 分区，把通用 ramdisk 与内核分离，配合 GKI（通用内核镜像）。

7.2 构建流水线

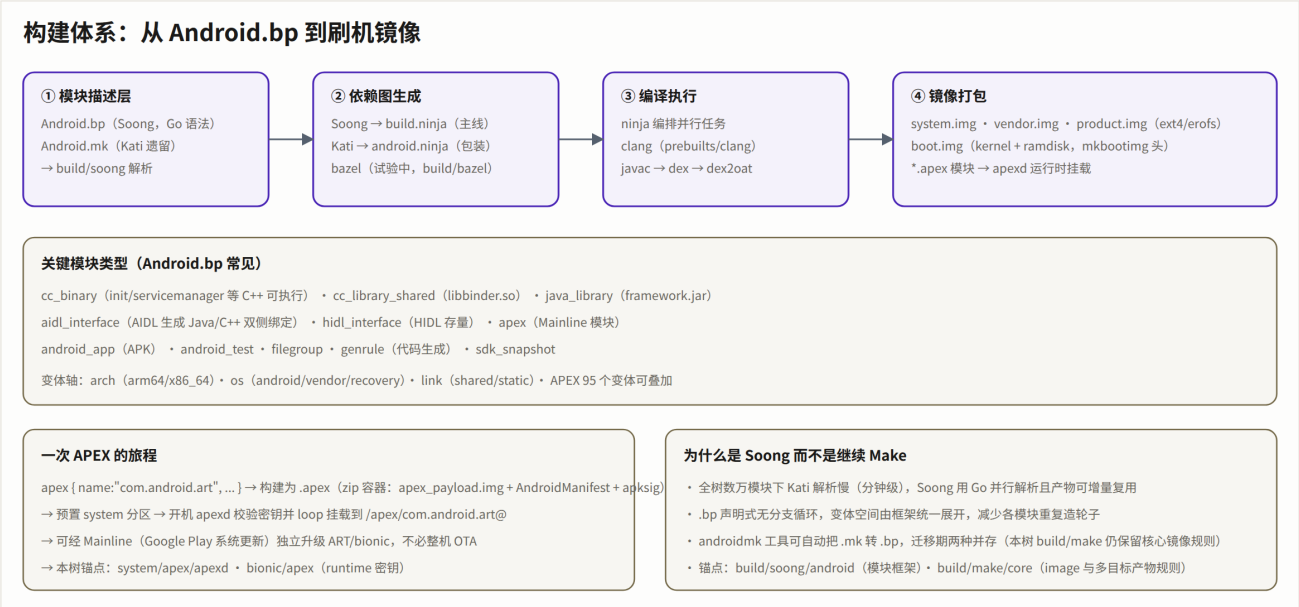


图 5 Soong/Kati→ninja→镜像的构建流水线与 APEX 生命周期、Soong 取代 Make 的动机

系统开发者的日常之一是读 `Android.bp`：`cc_binary` 产出守护进程，`aidl_interface` 从 .aidl 生成 Java/C++ 双侧绑定代码，`apex` 定义 Mainline 模块。**看一个模块的 Android.bp，就知道它编译成什么、装进哪个分区、暴露哪些接口**——这是改代码前的必做功课。

> **源码锚点 · 构建体系锚点**
>
> build/soong/android：Soong 模块框架（Go）
build/make/core/Makefile：system.img 等镜像组装规则（Kati 侧）
system/apex/apexd：APEX 运行时守护（挂载 /apex）
bionic/apex/com.android.runtime.*：runtime APEX 的 AVB 公钥与证书

7.4 从零构建一个系统 App（完整教程）

这是本册唯一一个端到端教程——从写代码到刷进设备。

第 1 步：写 App

text device/google/myproduct/apps/MyService/

```
├── Android.bp
├── AndroidManifest.xml
└── src/com/example/myservice/MainService.java
```

第 2 步：Android.bp

text Android.bp（完整模板）

```
android_app {
name: "MyService",
srcs: ["src/**/*.java"],
platform_apt: true, // 使用 platform 签名
privileged: true, // 安装到 priv-app
certificate: "platform", // platform 密钥
```

```
init_rc: ["MyService.rc"], // 随 App 安装 init 脚本
}
```

第 3 步：AndroidManifest.xml

xml AndroidManifest.xml（核心声明）

```
<manifest package="com.example.myservice"
android:sharedUserId="android.uid.system">
<application android:persistent="true"> <!-- 常驻 -->
<service android:name=".MainService"
android:exported="true" />
</application>
</manifest>
```

第 4 步：MyService.rc

text MyService.rc

```
service myservice /system/bin/app_process /system/bin com.example.myservice.MainService
class main
user system
group system
// 实际上 persistent App 由 AMS 启动，不走 init——
// 这里展示的是"非 persistent 版本"的 init 启动方式
```

第 5 步：声明到产品配置

text device/google/myproduct/device.mk

```
PRODUCT_PACKAGES += MyService
// 讲解注释：这一行告诉 Soong "构建时把 MyService 装进 system.img"
```

第 6 步：构建与部署

bash

```
source build/envsetup.sh
lunch aosp_x86_64-userdebug # 或你的产品 target
m MyService # 单模块构建（约 30 秒）
adb root && adb sync # 推送到设备
adb shell am startservice -n com.example.myservice/.MainService
# 或者看开机自启：
# adb shell getprop init.svc.myservice
```

第 7 步：验证

bash

```
adb shell dumpsys activity services com.example.myservice
# 应显示你的 Service 在运行
adb shell ps -A | grep myservice
# 应显示进程存在
```

设计洞察 · 逐段精读：这个教程串联了什么

第2步的 Android.bp 是书15 构建系统的入口；第3步的 sharedUserId=system 涉及书6 的权限；第4步的 .rc 是书2 init 语言的实践；第5步的 device.mk 是产品配置；第6步的构建命令是 Soong 增量。**一个简单 App 的部署，串联了丛书五册的核心知识**——这就是"系统开发者"与"应用开发者"的区别：你不仅知道怎么写 App，还知道它怎么被构建、部署、启动。

第八章 运行时体系：ART 与 Zygote

设计洞察 · 学习目标

1. 解释解释器/JIT/AOT 三级执行策略及 profile 的角色；2. 说明 boot.art 共享机器码与 Zygote 映射的关系；3. 描述 CC 收集器的 Region 与读屏障机制；4. 论证 App 进程必须从 Zygote fork 的原因



图 6 ART 三级执行策略（解释器/JIT/AOT + profile 反哺）与 zygote 进程模型、Dalvik→ART 演进动机

应用开发者视角的“Java”，在系统层是三件事：**dex 字节码规范、ART 虚拟机、Zygote 进程模型**。ART 的三级执行（解释器 → JIT → profile 引导的 AOT）让“安装快、启动快、运行快”这三个矛盾目标动态平衡。而 Zygote 把 `framework.jar` 与资源预加载进一个模板进程，之后所有 App fork 而出、共享只读页——第二册将从 `app_main.cpp` 开始走完这条链。

8.1 dex2oat 与启动镜像 boot.art

AOT 的输入输出由 `art/dex2oat` 定义：读入 .dex（vdex 中缓存原文）与 profile，产出 .odex（方法机器码）与 boot 镜像。**boot classpath**（framework.jar 等系统 jar 集）会额外编译为 `boot.art/boot.oat`——它们正是 Zygote 预加载时 mmap 的“共享机器码层”：所有 App 进程共享同一份框架类机器码映射，写时复制只在应用真正修改对象时发生。安装期与空闲期的编译策略由 `art/dex2oat/dex2oat.cc` 与 `system/serverconfiguration` 等协同控制；Cloud Profiles（Android

12+）让云端聚合的热点信息反哺本地 AOT。

8.2 ART GC 深潜：并发复制收集器

ART 的主收集器是**并发复制**（Concurrent Copying，CC）——art/runtime/gc/collector/concurrent_copying.cpp。这是 Android 性能的隐形基石。

为什么要并发?

text 停顿时间对比

```
Dalvik（标记-清除 STW）：200-500ms 停顿 → 13 年前的卡顿
ART 早期（标记-清除+部分并发）：50-100ms → 仍有感知
ART CC（读屏障并发搬移）：<5ms → 用户无感
// "并发"的意思：GC 与应用线程同时跑，只在小段时间暂停
```

Region Space：堆的分区管理

text RegionSpace（art/runtime/gc/space/region_space.cc）

```
┌────┬────┬────┬────┬────┬────┬────┬────┐
| R0 | R1 | R2 | R3 | R4 | R5 | R6 | R7 | ← 256KB 固定大小 Region
| 已用|空闲|已用|已用|大对|空闲|已用|空闲|
└────┴────┴────┴────┴────┴────┴────┴────┘
// 讲解注释①：Region 是分配和回收的基本单位
// 新对象在当前分配 Region 顶部指针碰撞分配（纳秒级）
// 回收时整个 Region 的存活对象搬走 → Region 直接回收
// 讲解注释②：大对象（>3 个 Region 大小）走独立的 LargeObjectSpace
```

读屏障：并发搬移的核心

text 读屏障（Read Barrier）的原理

```
每个对象的引用读取都被插桩：
if (obj.hasForwardingPointer()) {
obj = obj.getForwardedObject(); // 搬到新位置
self.reference = obj; // 修正引用
}
// 讲解注释③：这就是"用 CPU 换延迟"——
// 每条引用读取多几条指令（约 3-5% 性能开销）
// 但 GC 搬移对象不需要暂停全部线程
// 每个线程在自己的读屏障里"顺路"搬移
```

三种屏障与协作

屏障	触发时机	做什么
读屏障	每次读取引用	检查 forwarding 指针，按需搬移
写屏障	每次写入引用	记录跨 Region 引用（记忆集）
标记屏障	标记阶段	灰→黑转换时的标记传播

设计洞察 · 逐段精读：为什么面试常考 GC

GC 是"用户感知不到但影响每一帧"的机制。掉帧排查时 FrameTimeline 显示 "GC pause" 是常见原因。理解 CC 的工作原理后，你能从 dumpsys meminfo 的各 space 数据推断出 GC 行为（Region 剩余少=即将 GC、LargeObjectSpace 大=有内存泄漏的大数组）。**GC 不是黑盒，它是可推理的机器**。

源码锚点 · ART 关键产物与位置

art/dex2oat：AOT 编译器（读 dex+profile，产 odex）
/data/dalvik-cache：运行期编译产物目录
boot.art / boot.oat：框架类共享机器码（zygote 映射）
ZygoteInit.java:247 preloadClasses：预加载类清单的消费者（第二册）

设计洞察 · 为什么 App 进程必须从 Zygote fork

若每个进程自行加载 200MB 级的类与资源，启动要数秒且内存爆炸。fork + 写时复制让“加载”只发生一次；代价是所有 App 共享同一个 ART 运行时版本——这也是 Mainline 化 runtime APEX 的原因之一：运行时可独立升级，但升级必须整机重启生效。

第十章 深入属性系统：一次 setprop 的完整旅程

设计洞察 · 学习目标

1. 区分默认域/共享内存区/persist 域三个存储域；2. 走通写入路径的四道闸与 socket 凭证；3. 解释读路径零系统调用的实现；4. 用 on property 触发器联动 init 动作

属性系统是“最小的系统机制”，却五脏俱全——值得作为第一个逐函数精读的对象。

10.1 属性的三个存储域

- **默认域**：开机时 init 从 `/system/etc/prop.default`、`/vendor/build.prop`、`/data/property/persistent_properties` 等加载，见 `property_service.cpp:639` 的 `LoadProperties`。
- **共享内存区**：init 创建 `/dev/__properties__`，各域按前缀分区；所有进程 mmap **只读**。
- **persist 域**：`persist.` 前缀的属性由 `persistent_properties.cpp` 序列化到 `/data/property/persistent_properties`，重启不丢。

10.2 写入路径精读

任何进程调用 `SystemProperties.set`（bionic 的 `__system_property_set`）后发生的事：

cpp system/core/init/property_service.cpp:176（节选）

```
static uint32_t PropertySet(const std::string& name, const std::string& value,
std::string* error) {
```

```
size_t valuelen = value.size();
if (!IsLegalPropertyName(name)) { ... } // 1. 合法性：长度/字符集
if (valuelen >= PROP_VALUE_MAX) { ... }
if (StartsWith(name, "ctl.")) { // 2. ctl.* 是控制命令不是属性
return HandleControlMessage(name.c_str() + 4, value, getuid());
}
...
if (persistent_properties_loaded && StartsWith(name, "persist.")) {
error->insert(0, "Persistent property not loaded yet"); // 3. persist 未就绪前拒绝写
}
...
}
```

设计洞察 · 逐段精读：这段 30 行代码里的四道闸

①合法性（名字字符集/长度、值长度）——防注入与滥用；②ctl. 前缀劫持——属性写接口兼任服务控制通道（setprop ctl.start zygote 即可拉服务），但随后立即校验调用者权限；③persist 就绪检查——/data 未挂好前拒绝持久化，防止"丢在内存里装作持久化"；④ro. 只读——一次写后封死。四道闸合起来就是"一个最小 API 的完整安全叙事"。

三个细节值得一读：ctl. 前缀被劫持为**控制通道**（ctl.start/ctl.stop 直接驱动服务启停，等价于 setprop ctl.start zygote）；写入者身份（getuid）被记录；ro. 前缀只读属性在设置一次后返回 PREADONLY。

仲裁入口 HandlePropertySet（:481）先做 **SELinux source_context 检查**——不是任何进程都能写任何属性；通过 socket 收包的部分在 handle_property_set_fd（:527），它 accept 连接、读属性名与值、取对端凭证（SO_PEERCRED），再进入仲裁。**凭证来自内核，不可伪造**——这个模式 Binder 里将再次出现（第三册 tr.sender_euid）。

设计洞察 · 面试题常客：属性能跨 SELinux 域随便写吗

不能。属性按前缀划分 SELinux 上下文（property_contexts 文件），写 sys.powerctl 需要对应域授权。这也是为什么“重启设备”属性普通 App 写不了——策略层早已拒绝。

10.3 读取路径：为什么读不要系统调用

bionic 的 __system_property_find 直接在 mmap 的属性区里按前缀定位域、二分查找。属性区布局与查找算法在 bionic/libc/bionic/system_property_api.cpp 与 system/core/property_service/libpropertyinfoparser 中——读路径没有任何内核参与，这就是“纳秒级读”的来源。应用侧的 SystemProperties.java 只是 JNI 薄封装。

6.3 属性系统完整走读（从 set 到生效的每一跳）

这是全书最详细的一段单机制走读——它展示了"一个看似简单的 API 背后有多少协作"。

第零跳：App 调用

java App 进程

```
SystemProperties.set("debug.hwui.renderer", "skiagl");
// 或 bionic 层：__system_property_set(name, value)
```

第一跳：bionic 的 stub

cpp bionic/libc/bionic/system_property_api.cpp

```
// 讲解注释①：bionic 是 C 库（App 的 native 层），不走 Binder——
// 因为属性操作可能在 Binder 就绪前发生（init 第二阶段）。
// 改为写 socket：/dev/socket/property_service
int __system_property_set(const char* name, const char* value) {
// 组包 → sendto(prop_sock) → 等 init 回复
}
```

第二跳：init 的属性服务

cpp system/core/init/property_service.cpp:527

```
static void handle_property_set_fd() {
int s = accept4(property_set_fd, ...);
// 讲解注释②：SO_PEERCRED 取调用者凭证（pid/uid）——内核背书
// 讲解注释③：SELinux source_context 检查
// → HandlePropertySet(:481) → PropertySet(:176)
}
```

第三跳：仲裁与写入

cpp PropertySet 的四道闸（:176 起）

```
static uint32_t PropertySet(const std::string& name, ...) {
// 闸1：合法性（名字字符集/长度）
// 闸2：ctl.* 前缀劫持为控制命令
// 闸3：persist.* 前缀 → 持久化到 /data/property/
// 闸4：ro.* 只读属性 → 已设则拒绝
// 通过全部 → 写入共享内存属性区 → 通知等待者
}
```

第四跳：共享区与读取

text 所有进程的"读"路径

```
init 创建 /dev/__properties__（mmap 共享区）
→ 所有进程 mmap 只读 → 零系统调用读取
→ bionic __system_property_find 按 prefix 分区二分查找
// 讲解注释④：这就是"纳秒级读"的来源——
// 读操作完全在用户态内存完成，不进内核
```

属性触发（反向链）

text 属性变化如何触发系统动作

```
init.rc 里的 on property:sys.boot_completed=1
→ init 的 ActionManager 匹配触发器
→ 执行 action 里的命令（如 start 某服务）
// 这就是"属性即状态机"——
// ctl.start zygote 等价于 setprop ctl.start zygote
```

设计洞察 · 逐段精读：为什么这个走读重要

属性系统是**最简单的跨进程机制**（比 Binder 简单得多），但它展示了系统设计的所有核心模式：**凭证由内核采集**（SO_PEERCRED）、**策略仲裁集中**（init 单线程）、**读写不对称**（mmap 读+socket 写）、**声明式触发**（on property）。理解了这四个模式，后面学 Binder、学 SELinux 都会轻松——它们是同一个设计哲学的不同表达。

源码锚点 · 属性系统锚点速查

init.cpp:887 PropertyInit：第二阶段建立属性区
init.cpp:912 StartPropertyService：打开 /dev/socket/property_service
property_service.cpp:176 PropertySet：写仲裁（合法性/ctl./persist）
property_service.cpp:527 handle_property_set_fd：socket 收包与凭证
SystemServer 启动后 sys.boot_completed=1（AMS.java:5149，第二册详述）

第十一章 SELinux：系统的第二套权限体系

设计洞察 · 学习目标

1. 区分 DAC 与 MAC 各自回答的问题；2. 读懂 file/service/property 三张 contexts 表；3. 会写一个最小服务域的 .te 片段；4. 能把 avc denied 日志翻译成最小 allow 规则

传统 Linux 权限（DAC）回答“你是谁、文件给谁读写”；SELinux（MAC）回答“**你的域允许对什么类型做什么**”。Android 从 5.0 全量强制（enforcing），策略源码就在 `system/sepolicy/`。

11.1 从 init 的第一个动作看起

第二阶段 init 在属性初始化后立即做三件事（`init.cpp:901-903`）：`SelinuxSetupKernelLogging`、`SelabelInitialize`（加载 file_contexts）、`SelinuxRestoreContext`（`selinux.cpp:762`，按标签规则递归恢复全文件系统标签）。**在第一个服务启动前，规则就已就位**——此后每个服务进程 exec 时（`service.cpp` 的 `SetExecContext`）都会切入自己的 SELinux 域。

11.2 策略文件的分工

文件/目录	作用
`system/sepolicy/public`	导出给 vendor 使用的类型与接口（system_ext 同理）
`system/sepolicy/private`	system 私有策略
`system/sepolicy/vendor`	vendor 侧策略默认集

文件/目录	作用
*.te 文件	域规则：allow/init neverallow，如 allow init zygote_exec:file { read execute }
file_contexts	路径 → 文件类型标签（/system/bin/app_process64 → zygote_exec）
service_contexts	Binder 服务名 → 服务标签（activity → activity_service）
property_contexts	属性前缀 → 属性标签

读懂三张 contexts 表，是排查“服务起不来/属性写不进/设备节点打不开”类问题的钥匙。而 neverallow 规则是策略的“宪法条款”——编译期强制，谁也不能悄悄放水。

一个最小的服务域定义长什么样（示意，风格取自 system/sepolicy）：

text 服务域示意（.te 片段）

```
type mysvc, domain; # 声明域类型
type mysvc_exec, exec_type, file_type; # 可执行文件类型
init_daemon_domain(mysvc) # 由 init 拉起并切入域
allow mysvc activity_service:service_manager find; # 允许查询 activity 服务
neverallow mysvc shell_data_file:file { read write }; # 永久禁止碰 shell 数据
```

读 avc denied 日志的方法：avc: denied { find } for comm="mysvc" scontext=u:r:mysvc:s0 tcontext=u:object_r:activity_service:s0——按 scontext/tcontext 反查缺少的 allow 行，把最小权限补进策略。

设计洞察 · 系统开发者与 SELinux 的日常

新增系统服务必须：定义 .te 域 + service_contexts 注册 + rc 文件声明 selinux 域。忘了会看到 avc denied 日志（logcat -b events | grep avc）。调试期可 setenforce 0 临时全局放开（仅 userdebug），但**任何 denied 都应通过策略显式 allow 解决**，而不是永久关闭。

第十二章 图形与输入骨架概览

设计洞察 · 学习目标

1. 说出图形栈三角色（App/WMS/SF）的分工；2. 解释控制流与数据流分离的设计；3. 描述 VSync 的两路分发去向；4. 定位输入链路 InputReader/InputDispatcher 的源码骨架

渲染与输入是“系统开发者进阶深水区”，本册只画骨架坐标，细节留给后续批次。

图形一侧的三个角色：App 进程（frameworks/base 的 View 系统 + libhwui）把界面渲染进 BufferQueue 的图形缓冲；system_server 的 WMS 管理窗口层级与生命周期；surfaceflinger（frameworks/native/services/surfaceflinger）按 VSync 节拍把各窗口缓冲合成送显。三者之间传递的“货币”是 BufferQueue 里的 GraphicBuffer（DMA-BUF），控制流走 Binder，数据流走共享

内存——**控制与数据分离**是图形栈的核心设计。

输入一侧：`InputReader/InputDispatcher`（`frameworks/native/services/inputflinger`）从内核 evdev 读取事件，经 `system_server` 的 `InputManagerService` 与 WMS 的窗口焦点信息，把事件注入焦点窗口的 socket。App 侧 `InputEventReceiver` 收到后再分发给 View 树。

VSync 是整个图形栈的心跳。SurfaceFlinger 从 HWComposer 获得硬件 VSync，软件分发（DispSync/VsyncSchedule）给两路客户：App 侧的 Choreographer（`frameworks/base/core/java/android/view/Choreographer.java`，驱动 doFrame 绘制节奏）与 SurfaceFlinger 自身的合成节拍。“App 按节拍生产、SF 按节拍消费”的三缓冲流水线，是 60/120Hz 流畅的基础；渲染专册将沿 VSync 一路深挖。

> **源码锚点 · 图形/输入锚点**
> frameworks/native/services/surfaceflinger/SurfaceFlinger.cpp：合成主循环
frameworks/native/services/inputflinger/reader/InputReader.cpp：事件读取
SystemServer.java:1570：InputManagerService 创建（startOtherServices 阶段）
SystemServer.java:1586：WindowManagerService.main 启动点

第十三章 观测与调试：把系统打开给你看

> **设计洞察 · 学习目标**
> 1. 熟用四件套（service list/dumpsys/logcat/perfetto）；2. 掌握 boot_progress 启动观测点；3. 学会从 dumpsys 输出反查源码字段写点；4. 搭建单模块增量编译与实验环境

系统开发者的另一半功夫是**观测**。所有工具的原理都在源码里：

13.1 四件套

工具	看什么	源码原理
`adb shell service list`	已注册 Binder 服务	向 servicemanager（句柄 0）枚举 mNameToService
adb shell dumpsys <服务>	服务内部状态	调用服务的 dump() 方法——每个服务类自己实现（如 ActivityManagerService.dump）
`logcat`	全机日志	logd 汇聚 /dev/log/*，logcat 为客户端
`perfetto` / `systrace`	时间线与卡顿	atrace 埋点（ATRACE_CALL 宏遍布本丛书精读过的启动代码）

13.2 启动观测的具体落点

本丛书第二册的每个环节都有配套观测：`dmesg` 看 init 两阶段日志（"init second stage started!"，`init.cpp:839`）；`logcat -b events` 的 `boot_progress_*` 事件（如 `boot_progress_preload_start`，`ZygoteInit.java:887`）；`dumpsys activity services`、`dumpsys window` 验证服务与窗口状态；`sys.boot_completed` 属性作为“开机完成”的唯一权威信号（`ActivityManagerService.java:5149` 设置）。**先有观测，再谈优化**——这是系统性能工作的第一原则。

13.3 编译与实验环境

本树为完整可构建树。典型工作流：`source build/envsetup.sh` → `lunch`（选择目标，如 aosp_x86_64-userdebug）→ `m`（Soong 构建镜像）。虚拟设备 Cuttlefish（`device/google/cuttlefish`）可免真机做系统实验。日常单模块迭代用 `m services`（只编 services.jar）等目标 + `adb root; adb sync`。**本丛书只读研读，不改源码树**；但读者可以在自己的分支上练习。

`dumpsys` 的输出就是服务的自我陈述，例如：

text adb shell dumpsys power（节选）

```
POWER STATE (dumpsys power)
mWakefulness=Awake # 唤醒状态机当前值
mUserActivityTimeoutOverrideFromWindowManager=-1
mWakeLockSummary=0x1a # 活跃 wakelock 汇总
mUserActivitySummary=0x1
mHoldDisplaySuspendBlocker=true
mDisplay=Ready # 显示就绪标志
```

每个字段的赋值点都能在 `PowerManagerService.java` 中 grep 到——**dumpsys 输出即源码索引**，这是系统调试的高效路径：先 dumpsys 定位异常字段，再回源码找字段的所有写点。

（第十三章末的自检问题若都能回答，可径直进入第二册。）

> **设计洞察 · 自检问题（试试不回看正文回答）**
>
> 1) 五层各自只依赖哪一层？2) servicemanager 在哪个目录，为什么它不需要注册就能被找到？3) system.img 与 vendor.img 的接口契约由什么文件保证？4) 一个属性从 set 到生效经过哪些裁决？5) 为什么 App 进程必须是 fork 的产物？——答不上来的小节值得重读。

第十四章 进阶路线图：本丛书之后怎么走



图 7 从应用开发者到系统开发者的三阶段路线、配套技能栈与自测里程碑

路线图给了三阶段与里程碑，这里补三句话原则：**第一，主线优先于专题**——先走完启动与 Binder 两条主线，再挑专题深潜，否则专题知识挂不住。**第二，输出倒逼输入**——每读完一册，合上书画一遍时序图/调用链，画不出的地方就是没懂的地方。**第三，让源码树成为日常**——把 riagrep 检索、dumpsys 验证变成肌肉记忆，系统开发者的核心竞争力不是背下架构，而是“面对陌生问题时的定位速度”。

丛书后续批次（AMS 与四大组件、WMS 与渲染栈、PMS 与安装流程）将沿用本册的写法：真实源码锚点 + 执行链路 + 设计决策。

第十五章 设计决策分析（全书总结）

本章把散落各章的“为什么”收拢成十条可复用的架构判断：

- **分层与依赖方向**：每层只依赖下一层，跨层只能走公共通道（Binder/属性）。反过来“哪层都不属于”的代码，就是放错位置的代码。
- **Binder 单一 IPC 抽象**：统一的安全模型（内核验 UID/PID）胜过多个“更快”但各搞一套安全机制的通道。
- **HAL 三代演进**：隔离（legacy→HIDL）→ 简化（HIDL→AIDL）。隔离是原则，双栈是代价，归一是终局。
- **分区即解耦**：Treble 用物理分区强制接口稳定，让 system 升级与 vendor 解耦；GSI/GKI 是同一思想的镜像/内核版。
- **APEX 模块化**：把“必须随系统走但又想独立升级”的部分（ART、bionic）打包成可挂载的 mini 分区。

- **system_server 集中 vs 关键服务独立**：集中减少 IPC 与状态同步成本；SurfaceFlinger/audioserver 独立是为崩溃隔离与实时性——边界画在“故障域”上。
- **SystemServiceRegistry 门面**：进程内缓存 Binder 代理，屏蔽服务获取细节，App/系统应用同一入口。
- **属性系统的不对称设计**：读极简（mmap）、写集中（init 仲裁），适配最早期的运行环境。
- **Zygote 预加载 + COW**：以一次加载换千百次 fork 的速度与内存，是 Android 多进程模型成立的前提。
- **SELinux 强制访问控制**：DAC 会 root 绕过，MAC 不允许；策略与代码同树（system/sepolicy），改服务必查策略。

第十六章 深水区专题一：SELinux 策略实战与 APEX 内部

> **设计洞察 · 学习目标**
> 1. 按四步动作给新服务补齐策略；2. 理解 neverallow 的编译期宪法地位；3. 走通 apexd 激活与回滚状态机；4. 用口诀判断模块化边界

16.1 SELinux 策略编写实战（读改两件套）

策略源码在本树的体量会刷新你的认知：仅 `system/sepolicy/private/` 下就有 **202 个 .te 文件**。编写实践的三板斧：

text 给新服务 mysvc 补策略的标准动作

```
① 定义域：mysvc.te 写 type mysvc, domain; type mysvc_exec, exec_type, file_type;
+ init_daemon_domain(mysvc)
② 声明清单：file_contexts 加 /system/bin/mysvc u:object_r:mysvc_exec:s0
service_contexts 加 mysvc u:object_r:mysvc_service:s0
③ 最小放行：按 avc denied 逐条补 allow——
neverallow 是"宪法"（domain.te:119/:137 起大量出现）：
编译期硬校验，任何 allow 与其冲突直接拒绝构建
④ 应用侧打标：seapp_contexts（sepolicy/private/seapp_contexts）
按 package name/uid 给 App 域（untrusted_app 等）
```

逐段精读：`audit2allow < avc.log` 能自动生成候选 allow，但**永远不要无脑采纳**——它只会"让报错消失"，不知道语义该不该放行。生产实践是先看 neverallow 是否已经替你拒绝了这条路（例如禁止普通域写 shell_data_file），再决定是改架构还是补权限。

16.2 APEX 激活与回滚内部（apexd）

源码锚点 · apexd 关键文件

system/apex/apexd/apexd_main.cpp:99 main 入口（init 的 post-fs-data 期拉起）
apex_file_repository.cpp：收集 /system/apex 与 /data/apex（会话安装位）
apex_database.cpp：活跃 APEX 挂载表（loop 设备与 /apex 视图）
apexd_session.cpp：staged 会话与激活/回滚状态机

激活流程：开机 apexd 扫描 → 校验 AVB 公钥与版本 → loop 挂载到 `/apex/<name>@<ver>` → 类路径（apex_classpath.cpp）注入 ART。**回滚**：staged 会话默认"试运行"（boot flag），连续两次启动失败自动退回旧版——与会话状态机（session state：staged→activated→reckoned）一体，这就是 Mainline"敢独立发版"的底气。

16.3 Mainline 模块边界速查（哪些已模块化）

模块	内容	升级通道
com.android.art	ART 运行时与 dexopt（书6 §3.3 的 artd）	Google Play 系统更新
com.android.runtime	bionic 与运行时库	同上
com.android.tethering	网络共享与连接模块	同上
com.android.permission	PermissionController（权限 UI/逻辑）	同上
com.android.scheduling	JobScheduler 域逻辑	同上
com.android.wifi	Wi-Fi 栈（部分）	同上
com.android.media / mediaprovider	媒体编解码配置/存储提供者	同上
（对照）framework/system 分区本体	AMS/WMS/PMS 等核心服务	仅整机 OTA

判断口诀：**"策略与 UI"可进模块，"事实与账本"留在系统**（与书6 权限对话框同一条分界线）。

第十七章 深水区专题二：Soong 实战与链接器命名空间

设计洞察 · 学习目标

1. 写出 cc_binary 的完整属性心智；2. 理解 Soong 变体轴的展开逻辑；3. 解释 linker namespace 的隔离层次；4. 按场景选对 sanitizer 组合

17.1 写一个 Android.bp 的完整心智

text 给书4 的假设服务 mysvc 写构建描述（Soong 语法要点）

```
cc_binary { // 产物：/system/bin/mysvc
name: "mysvc",
srcs: ["mysvc.cpp"],
shared_libs: ["libbinder", "libutils", "libcutils"], // 依赖→dt_needed
init_rc: ["mysvc.rc"], // 随二进制安装 rc（书2 的解析入口！）
vintf_fragments: ["mysvc.xml"], // HAL 时必须：VINTF 清单
relative_install_path: "hw", // 装到 /system/bin/hw（HAL 惯例）
proprietary: true, // 归入 vendor 分区（对应 soong 配置）
stem: "vendor.mysvc-default", // HAL 服务命名惯例
}
```

变体轴的直觉：Soong 会按 arch（arm64）×os（android/vendor）×link（shared/static）自动展开出多个构建目标——**你声明的是"家族"，构建系统生的是"全家福"**。改 bp 后 `m module-info` 可查模块间依赖图。

17.2 链接器命名空间与 linkerconfig

Android 的动态链接不是"一锅端"：**linker namespace**（配置源 `system/linkerconfig/`）按进程把 so 划入不同命名空间——vendor 进程只能见 vendor 命名空间导出的符号、framework 的 libbinder 与 vndbinder 版本互不可见（Treble 在**链接期**的隔离，比运行期 SELinux 更早）。bionic 与 glibc 的差异清单：缺 NSS/PAM、自带属性/日志 API、ABI 随 NDK 冻结——读码时遇到 `__system_property_get` 这类"陌生 libc 函数"，答案都在 bionic。

17.3 sanitizer 家族（内存安全工具链）

ASAN（插桩+影子内存，构建开关 sanitize: {address: true}）、HWASAN（ARMv8 TBI 硬件标签，内存开销小得多）、**MTE**（内存标签硬件化，书2 forkSystemServer 里 `arm64.memtag` 属性即开它）、GWP-ASan（低概率采样，生产期兜底）。系统服务默认带 GWP-ASan 抽奖（SystemServer.java:763 的 GWP_ASAN_LEVEL_LOTTERY 同源思想）——**内存安全不是单一工具，是分层的"概率×覆盖"组合拳**。

第十八章 边境机制三讲：JNI、Context 家族与 native 智能指针

设计洞察 · 学习目标

1. 区分两种 JNI 注册与三类引用的生命周期；2. 分清 Context 家族三支与各自的令牌能力；3. 掌握 sp/wp/RefBase 的计数语义；4. 在 Binder 框架代码里认出这些机制的现场

前三册反复跨过 Java/Native 边境却未教过"过境规则"。本章补上三门必修前置课。

18.1 JNI：两种注册与三类引用

注册：静态（Java 声明 native、库加载时 `RegisterNatives` 显式绑定——框架全部走此路，`AndroidRuntime.cpp` 的 startReg 一次注册几百个）与动态（`JNI_OnLoad` 里按签名查找，JNI 调试器友好但慢）。**引用三态**：局部引用（方法栈内，帧返回即回收——**循环里不释放会撑爆表**）、全局引用（跨线程/跨调用持有，`NewGlobalRef/DeleteGlobalRef` 成对）、弱全局（不阻止 GC，用前必须 `GetObjectLocalRef` 升格检查）。第三册 `javaObjectForIBinder` 里 BinderProxy 的 nativeData 长期持有就是全局引用的现场；`JavaBBinderHolder` 的弱缓存（:462-485）则是弱引用的现场。**异常语义**：JNI 异常不中断 native 执行，必须显式 `ExceptionCheck` 后返回——书3 的 transact JNI 里每一跳都有这个检查。

18.2 Context 家族：一把钥匙配千把锁

Context 是"访问系统环境的句柄"，族谱三支：**ContextImpl**（`core/java/android/app/ContextImpl.java:192`，唯一实现，持有 LoadedApk/权限主题/显示区）——**ContextWrapper**（纯委托壳，Service/Application 都是它的子类）——以及各组件 Context（Activity 的还带窗口与主题注入）。系统侧的"系统 Context"（书2 createSystemContext :882）是给 system_server 里那些"没有界面的应用"配的 ContextImpl。理解两件事就够用：**getSystemService 的缓存挂在 ContextImpl 上**（书1 §5.2 的 fetcher 缓存）；**startActivity 的"令牌"来自 Context 背后的窗口/Activity**——用 Application Context 启动有 UI 依赖的 Activity 会失败，就是令牌缺失的经典报错。

18.3 sp/wp/RefBase：native 世界的 GC

libutils 自带一套引用计数 GC（`system/core/libutils/include/utils/RefBase.h:281`）：**sp<T>** 强指针（构造 incStrong/析构 decStrong，计数归零销毁）、**wp<T>** 弱指针（只升弱计数，-promote() 升格检查对象是否存活——**native 世界的"软引用"**）。为什么不用裸指针：Binder 世界对象跨线程跨进程共享（书3 的 BBinder/BpBinder 全是 RefBase 子类），需要确定性生命周期；为什么不用 std::shared_ptr：历史先于标准库成熟，且弱引用语义可自定义。第三册的 BC_ACQUIRE/BC_INCREFS 命令，本质是**把这套计数跨越进程镜像**——两端各自 sp，驱动保证两本账同步。读懂 RefBase，书3 §5.4 的引用计数命令族就从"名词"变成"机制"。

> **源码锚点 · 本章锚点**
>
> frameworks/base/core/jni/AndroidRuntime.cpp startReg（静态注册大表）
core/java/android/app/ContextImpl.java:192 / LoadedApk.java:107
system/core/libutils/include/utils/RefBase.h:281
案例：android_util_Binder.cpp:749（全局引用）/:462（弱缓存）

附录 A 版本指纹与阅读路线

版本指纹（`build/make/core/version_defaults.mk`）：`DEFAULT_PLATFORM_VERSION := TP1A`；`PLATFORM_SDK_VERSION := 33`；`PLATFORM_SECURITY_PATCH := 2022-08-05`。即 Android 13 Tiramisu 初始版本。

下一步：第二册《系统启动主线》将从内核拉起 init 的第一行代码开始，沿 `init → Zygote → SystemServer → Launcher` 逐函数走完开机全程；第三册《Binder IPC 全链路剖析》将把本册反复出现的“经 Binder 调用”逐跳展开。三册读完，你面对任何 Android 系统问题，都能定位到“哪一层、哪个进程、哪条调用链”。

附录 B 系统进程归属速查

进程	pid 典型	出生方式	一句话职责
init	1	内核启动	万物之父，rc 解释器与服务监护
ueventd	2	init fork（argv[0] 切换）	uevent 处理、/dev 节点与固件加载
servicemanager	-	init 服务	Binder 服务名录
zygote（主/辅）	-	init 服务	App 与 system_server 之母
system_server	-	zygote fork	160+ 框架服务宿主
surfaceflinger	-	init 服务	图形合成
audioserver	-	init 服务	音频策略与混音
cameraserver	-	init 服务	相机 HAL 宿主
mediaserver	-	init 服务	媒体编解码
netd	-	init 服务	网络配置执行者
vold	-	init 服务（early-fs）	存储卷与加密
lmkd	-	init 服务	低内存查杀
logd	-	init 服务	日志中枢
bootanimation	-	init 服务（core animation）	开机动画
adbd	-	init 服务	调试桥（user 版受限）
installd	-	init 服务	APK 安装/dexopt 执行者
*.vendor HAL 进程	-	init 服务（class hal）	硬件服务（vendor 分区）

附录 C 常用系统属性速查（带源码出处）

属性	含义	设置/读取位置
ro.build.version.sdk	SDK 版本（33）	构建期写入，build.prop
ro.product.cpu.abilist	CPU ABI 列表	决定 zygote 单/双形态
ro.bootmode	启动模式（charger=充电）	init.cpp:978 分支触发链
ro.crypto.state	加密状态（encrypted/unencrypted）	zygote-start 触发条件
sys.boot_completed	开机完成权威标志	AMS.java:5149 设置
init.svc.zygote	服务状态机（running/restarting/stopped）	service.cpp:684 NotifyStateChange
persist.sys.timezone	时区	SystemServer.run:777 校正
ctl.start / ctl.stop	服务控制命令	property_service.cpp:185 分流
debug.sf.*	SurfaceFlinger 调参	SF 读取
dalvik.vm.*	ART 参数（heapsize 等）	libart 读取

附录 D 术语表

术语	展开	一句话解释
AOSP	Android Open Source Project	Android 开源项目及其源码树
ART	Android Runtime	5.0 起的运行时，替代 Dalvik
APEX	Android Pony EXpress	可独立升级的模块容器（.apex）
AVB	Android Verified Boot	镜像完整性校验链
CTS	Compatibility Test Suite	兼容性认证测试集
GKI	Generic Kernel Image	通用内核镜像（init_boot 分离配套）
GSI	Generic System Image	可刷的通用 system 镜像
HAL	Hardware Abstraction Layer	硬件抽象层
HIDL	HAL Interface Definition Language	Treble 时代的 HAL IDL（冻结维护）
JIT/AOT	Just/Ahead Of Time	即时/提前编译
LMKD	Low Memory Killer Daemon	用户态低内存查杀

术语	展开	一句话解释
MAC/DAC	Mandatory/Discretionary Access Control	强制/自主访问控制
PMS	PackageManagerService	包管理服务
PSI	Pressure Stall Information	内核压力信息（lmkd 输入）
RC	Android Init Language	.rc 配置文件（init 语言）
SEPolicy	SELinux Policy	强制访问控制策略
Treble	Project Treble	system/vendor 架构解耦工程
USAP	UnSpecialized App Process	Zygote 预孵化进程池
VINTF	Vendor Interface	设备接口清单与兼容矩阵
WMS	WindowManagerService	窗口管理服务

附录 E 上手实验（半天量）

实验一“服务普查”：`adb shell service list` 统计服务总数，找出十个你认识的与十个陌生的；陌生的逐一 `dumpsys` 看自述，猜职责再回源码验证（services/core 对应目录）。

实验二“属性观察”：`adb shell getprop | wc -l` 数属性量级；`getprop init.svc.zygote`、`getprop sys.boot_completed`、`getprop ro.build.version.sdk` 各自对应本册哪一章；用 `setprop` 试试写 `ro.` 前缀，观察拒绝行为（property_service.cpp:176 的只读检查）。

实验三“SELinux 初体验”（userdebug）：`adb shell getenforce` 看模式；`setenforce 0` 前后各跑一次某功能，`logcat -b events | grep avc` 收集差异；把一条 denied 翻译成策略语言（对照第十一章 11.2 的表）。

实验四“目录定向”：不看本册，凭记忆说出 frameworks/native/cmds/servicemanager、system/core/rootdir、frameworks/base/services/core/java/com/android/server/wm 各住着什么；再全书检索验证。

每个实验写三行结论：看到了什么、对应哪条源码锚点、哪里出乎意料。

第十九章 系统服务注册全流程走读：一个服务的五跳人生

> **设计洞察 · 学习目标**
>
> 1. 复述一个系统服务从实例化到可被调用的五跳；2. 说出 servicemanager addService 的七道闸；3. 解释 getService 的进程内缓存与 lazy 服务按需拉起；4. 用 service list 与 dumpsys 反向定位注册失败

第五章 5.5 节的"服务全表"是一张鸟瞰图：160+ 服务、按启动批次分组、每行一句话职责。但名单不等于机制——**表里的一个服务，究竟是怎么从一行 Java 代码变成全机可调用的 Binder 对象的？**本章把镜头推进到微观，选一个真实服务：振动服务 VibratorManagerService（Binder 名 vibrator_manager）。它是"标准中的标准"：有 Lifecycle 壳、有 boot phase 依赖、注册路径与第五章表格里的绝大多数服务完全同构。走完本章的五跳，"服务全表"对你就不再是名单，而是同一模式的 N 次重复。

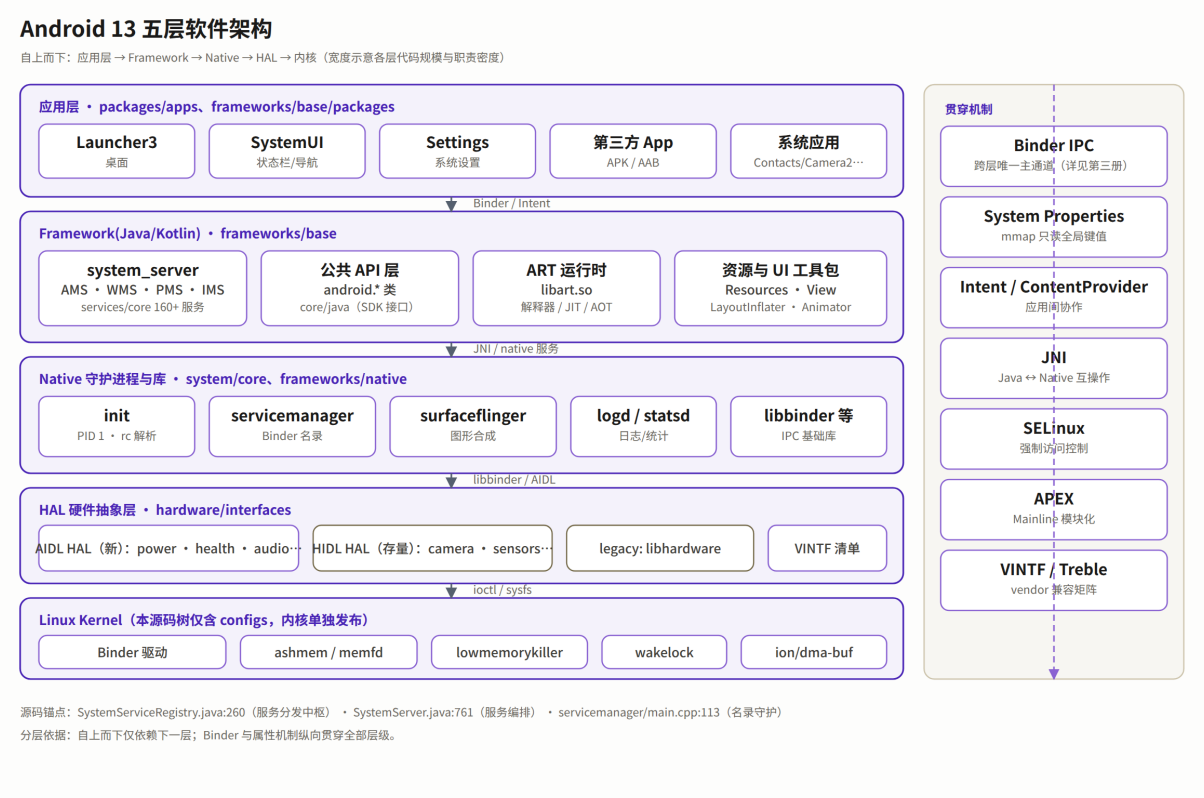


图 8 服务注册五跳恰好纵贯三层：Framework 的 SystemServer/SystemServiceManager、Native 的 servicemanager 与 libbinder、内核的 Binder 驱动——本章为每一跳给出源码锚点

链路位置：这一章是全书三份知识的交汇点——第五章的服务版图（静态名单）、第六章的层间通信（Binder 通道）、第十一章的 SELinux（注册仲裁）。你即将看到：一次"注册"= 一次跨进程 Binder 调用（书3 的第一份见面礼）。

19.1 第一跳：SystemServer 里的一行注册代码

system_server 启动到 `startOtherServices`（第五章 Other 批，:1404 起）后，注册振动服务只占三行：

java frameworks/base/services/java/com/android/server/SystemServer.java:1543-1545（注册现场）

```
t.traceBegin("StartVibratorManagerService");
// 讲解注释①：traceBegin/traceEnd 是 perfetto 时间线的埋点——
// 第十三章说过"先有观测"，这一对括号就是开机动线图上 StartVibratorManagerService 段的来源
mSystemServiceManager.startService(VibratorManagerService.Lifecycle.class);
// 讲解注释②：注意注册的是 Lifecycle 壳类，不是服务本体——
// SSM 需要一个统一暴露 onStart/onBootPhase 钩子的"生命周期外壳"，
```

```
// 这是与"直接 new + addService"的写法分野的关键（对比见下）
t.traceEnd();
```

紧随其后数行（SystemServer.java:1547-1549）就能看到另一种写法：`dynamicSystem = new DynamicSystemService(context); ServiceManager.addService("dynamic_system", dynamicSystem);`——**简单服务直接实例化注册，有生命周期/启动期依赖需求的服务走 Lifecycle 工厂**。第五章表格里的服务分属这两种风格，判断标准是它需不需要 boot phase。

19.2 第二跳：SystemServiceManager 的反射工厂

`startService(Class)` 做两件事：反射创建、登记启动。这是理解"为什么系统服务必须带 Context 构造器"的地方：

java
frameworks/base/services/core/java/com/android/server/SystemServiceManager.java:207-227、243-262（骨架）

```
// 讲解注释①：第一段——反射创建（:207-227）
final Constructor<T> constructor = serviceClass.getConstructor(Context.class);
service = constructor.newInstance(mContext);
// 所有系统服务的公共约定：public 构造器 + Context 参数。
// 七种反射异常全部包成 RuntimeException 抛出——任何一个服务起不来，
// startOtherServices 直接崩溃，system_server 退出，zygote 重启整机（第二册"自杀式容错"）
// 讲解注释②：第二段——登记与启动（:243-262）
mServiceClassnames.add(className); // 防重复启动（同名类只起一次）
mServices.add(service); // 登记进 mServices——后续 boot phase 广播的遍历对象
service.onStart(); // 立即调用——onStart 只做"发布"，不做跨服务依赖
// 讲解注释③：为什么 onStart 里禁止依赖别的服务？因为 Other 批是顺序执行的，
// 在你之前启动的服务不保证"已就绪"（ready），只保证"已发布"（published）。
// 真正的依赖协调走 onBootPhase——SystemService.java:77-121 定义了 8 个相位
//  （PHASE_WAIT_FOR_DEFAULT_DISPLAY=100 到 PHASE_BOOT_COMPLETED=1000）
```

设计洞察 · 逐段精读：onStart 与 onBootPhase 的分工

onStart 回答"我是谁"（构造 + publishBinderService）；onBootPhase 回答"现在几点"（系统就绪到什么程度）。VibratorManagerService.Lifecycle（:117-121）就是教科书示范：onStart 里 new + 发布，onBootPhase 收到 PHASE_SYSTEM_SERVICES_READY（500）才调 systemReady()。**把重依赖放进 onStart 是新手的经典启动崩溃来源**——它可能在依赖服务尚未注册时被调用。

19.3 第三跳：onStart 与 publishBinderService

java frameworks/base/services/core/java/com/android/server/vibrator/VibratorManagerService.java:104-122（Lifecycle，骨架）

```
public static class Lifecycle extends SystemService {
private VibratorManagerService mService;
public Lifecycle(Context context) { super(context); }
@Override
public void onStart() {
// 讲解注释①：真正的服务本体在这里才被创建
mService = new VibratorManagerService(getContext(), new Injector());
// 讲解注释②：publishBinderService（SystemService.java:523-535）是
// ServiceManager.addService(name, service) 的 protected 包装——
```

```
// "发布"一词意味着：从这一刻起，Binder 名存在了，但跨进程还需要下一跳
publishBinderService(Context.VIBRATOR_MANAGER_SERVICE, mService);
}
@Override
public void onBootPhase(int phase) {
if (phase == SystemService.PHASE_SYSTEM_SERVICES_READY) {
// 讲解注释③：phase 500 时依赖的服务都已发布，才敢做 systemReady
mService.systemReady();
}
}
}
```

第三跳结束在进程边界上：`mService` 是 system_server 里的一个普通 Java 对象（IBinder 的 Stub 端），**它还没有被任何其他进程知道**。跨过这条边界，就是第四跳。

19.4 第四跳：跨进程注册——servicemanager 的七道闸

java frameworks/base/core/java/android/os/ServiceManager.java:179-213（addService 链，骨架）

```
public static void addService(String name, IBinder service, boolean allowIsolated, int
dumpPriority)
{
// 讲解注释①：getIServiceManager()（:122）是懒加载单例——首次调用时
// BinderInternal.getContextObject() 经 JNI（android_util_Binder.cpp:1141）
// 走到 ProcessState::getContextObject → getStrongProxyForHandle(0)（ProcessState.cpp:146）
// 句柄 0 就是 servicemanager 的约定地址——它自己不需要被注册，"约定即地址"
getIServiceManager().addService(name, service, allowIsolated, dumpPriority);
// 讲解注释②：这是一次真正的 Binder 事务：system_server（客户端）
// → Binder 驱动（内核层，一次拷贝）→ servicemanager（Native 层服务端）
// 书3 将把这一次事务拆到 binder_ioctl 的每一行
}
```

事务到达 **servicemanager**（`frameworks/native/cmds/servicemanager/ServiceManager.cpp:295-341`）后，注册要过**七道闸**——这是"服务找不到"类问题的全部根因库：

闸	源码位置	拒绝条件	典型报错
1	ServiceManager.cpp:300-303	调用者是 App UID（appid 大于等于 AID_APP）	App UIDs cannot add services
2	:304-306	SELinux 策略未授权 canAdd	SELinux denial（EX_SECURITY）
3	:309-312	binder 为 null	EX_ILLEGAL_ARGUMENT
4	:313-317	名字非法（超 127 字符或含非法字符）	Invalid service name
5	:320-324	HAL 服务未在 VINTF 清单声明	VINTF declaration error
6	:327-332	linkToDeath 失败（无法感知服务进程死亡）	EX_ILLEGAL_STATE

闸	源码位置	拒绝条件	典型报错
7	:335-341	—（通过后覆盖写 mNameToService 并回调 onRegistration）	—

> **设计洞察 · 逐段精读：七道闸的排序有讲究**
> 闸 1 把 App UID 挡在最外——**普通应用永远无权注册服务**，这是 Binder 服务名录的准入底线；闸 2 是 SELinux（书11 的 service_contexts 在这里被消费）；闸 5 是 Treble 的 VINTF（第三章文件六）在运行期的回响——**构建期对齐过的清单，注册期再验一遍**。闸 7 的"覆盖写"说明 addService 是允许覆盖的，同名服务后来者居上——排查"调到的服务行为不对"时，先确认没有两个进程在抢注同名。

19.5 第五跳：客户端 getService——缓存、查询与 lazy 拉起

注册的对偶是获取。任何进程里 `context.getSystemService(Context.VIBRATOR_MANAGER_SERVICE)` 最终都收敛到：

java frameworks/base/core/java/android/os/ServiceManager.java:141-154、369-372（getService 链，骨架）

```
public static IBinder getService(String name) {
IBinder service = sCache.get(name);
// 讲解注释①：sCache（:52，每进程一份 ArrayMap）——首次获取后缓存 Binder 代理，
// 这正是第五章 5.2 SystemServiceRegistry"管理器对象是缓存马甲"的另一层缓存：
// Registry 缓存管理器，ServiceManager 缓存 IBinder 代理
if (service != null) { return service; }
return Binder.allowBlocking(rawGetService(name));
// 讲解注释②：allowBlocking——框架服务默认可能阻塞（服务端可能在忙），
// 对 framework 服务的调用默认按阻塞调用处理（书3 讲 oneway 与阻塞语义）
}
private static IBinder rawGetService(String name) throws RemoteException {
final IBinder binder = getIServiceManager().getService(name); // :372
// 讲解注释③：又一次 Binder 事务到达 servicemanager：ServiceManager.cpp:232 getService
// → tryGetService（:245-283）依次做三件事：
// ① mAccess->canFind(ctx, name)——SELinux service_manager find 权限
// ② allowIsolated 检查——未开放隔离的服务对 isolated 进程隐身
// ③ 找不到且 startIfNotFound → tryStartService(:543)
if (binder == null) { throw new IllegalStateException("Delayed service " + name + ...); }
return binder;
}
```

`tryStartService`（ServiceManager.cpp:543-553）是本章最巧的一笔：servicemanager 自己**无权也不愿**拉起进程，它把"服务不存在"翻译成 **setprop ctl.interface_start aidl/服务名**——通过第十章的属性控制通道委托 init 去启动声明为 lazy 的 AIDL 服务进程。**第六章的属性系统在这里反哺 Binder**：两条机制的握手点就在这一行。

设计洞察 · 设计洞察：getService 为什么设计成"查不到先拉起"

对硬件厂商而言，HAL 服务常驻内存是浪费——lazy 服务按需启动、无人使用时退出。servicemanager 的 tryStartService 让"客户端永远拿到服务"成为不变量，把按需启动的复杂度从每个调用方挪进了服务名录本身。代价是：getService 可能从纳秒级变成秒级（要等进程起来）——**这就是为什么 Java 侧先查 sCache，也是为什么关键框架服务都常驻 system_server 而不走 lazy**。

19.6 观测验证：三条命令读活注册表

第十三章的四件套在注册场景的落点：

text 注册表观测三连（可直接执行）

```
adb shell service list | grep vibrator
// 讲解注释①：底层走 servicemanager 的 listServices（ServiceManager.cpp:350 起），
// 同样要过 mAccess->canList 的 SELinux 检查——shell 域有授权，普通 App 没有
adb shell dumpsys vibrator_manager
// 讲解注释②：注册成功不等于服务健康——dumpsys 调用的是服务自己的 dump() 实现，
// 能输出自述说明 system_server 侧的 VibratorManagerService 还活着
adb shell getprop init.svc.zygote && adb logcat -d | grep "Starting
com.android.server.vibrator"
// 讲解注释③：SSM 在 startService 里 Slog.i("Starting " + name)（:210）——
// 每个服务注册都会留下这条日志，是判断"注册到没到"的第一现场
```

跨册交叉引用：书2 将给出 startOtherServices 的完整启动时序与 8 个 boot phase 的广播顺序；书3 将把本章的两次 Binder 事务（addService/getService）拆解到 Parcel 组包、驱动 ioctl、BBinder onTransact 的每一跳；书6/书14 承接闸 5 的 VINTF 清单体系；书16 承接闸 2 的 service_contexts 策略。本章排障落点见排障手册"场景二：服务找不到"。

第二十章 一次点击的全程旅行：从指尖到像素的九跳（跨册整合）

设计洞察 · 学习目标

1. 列出触摸事件从硬件到屏幕的九跳及各自所在进程；2. 解释输入为什么走 socket 而不是 Binder；3. 解释 Reader/Dispatcher 双线程分工与 ANR 计时点；4. 用 perfetto 在时间线上对号每一跳

本册前十八章是横切面的地图；本章做一次**纵贯线**：一根手指按下去，到屏幕上一个按钮高亮，中间发生了什么？这条线贯穿触摸硬件、内核、inputflinger、App、渲染线程、SurfaceFlinger 与显示控制器，共九跳——每一跳标注所在进程、关键源码文件与深读它所在的分册。它是全书各章知识的"总装现场"：第五章的服务分发、第六章的通信机制选型、第十二章的图形输入骨架、第八章的运行时，都在这条线上各就各位。

20.1 九跳总表：先看全程，再逐跳落地

跳	发生什么	所在进程	关键源码锚点	深读分册
1	触摸屏采样上报	触摸 IC 固件	驱动在本树外（UAPI 头 external/kernel-headers）	书14
2	内核 evdev 生成事件	内核	drivers/input（本树外）	书14
3	EventHub 读 /dev/input	system_server	inputflinger/reader/EventHub.cpp:1533	书5
4	InputReader 加工成 MotionEvent	system_server	reader/InputReader.cpp:81	书5
5	InputDispatcher 定窗与决策	system_server	dispatcher/InputDispatcher.cpp:1990	书5
6	socket 注入焦点窗口	system_server 到 App	libs/input/InputTransport.cpp:334	书5
7	App 消费事件并请求绘制	App	ViewRootImpl.java:8819/6542	书5
8	Choreographer 节拍渲染	App 与 RenderThread	Choreographer.java:742、HardwareRenderer.java:456	书8
9	合成送显	surfaceflinger	SurfaceFlinger.cpp:1992/2164、HWComposer.cpp:508	书8

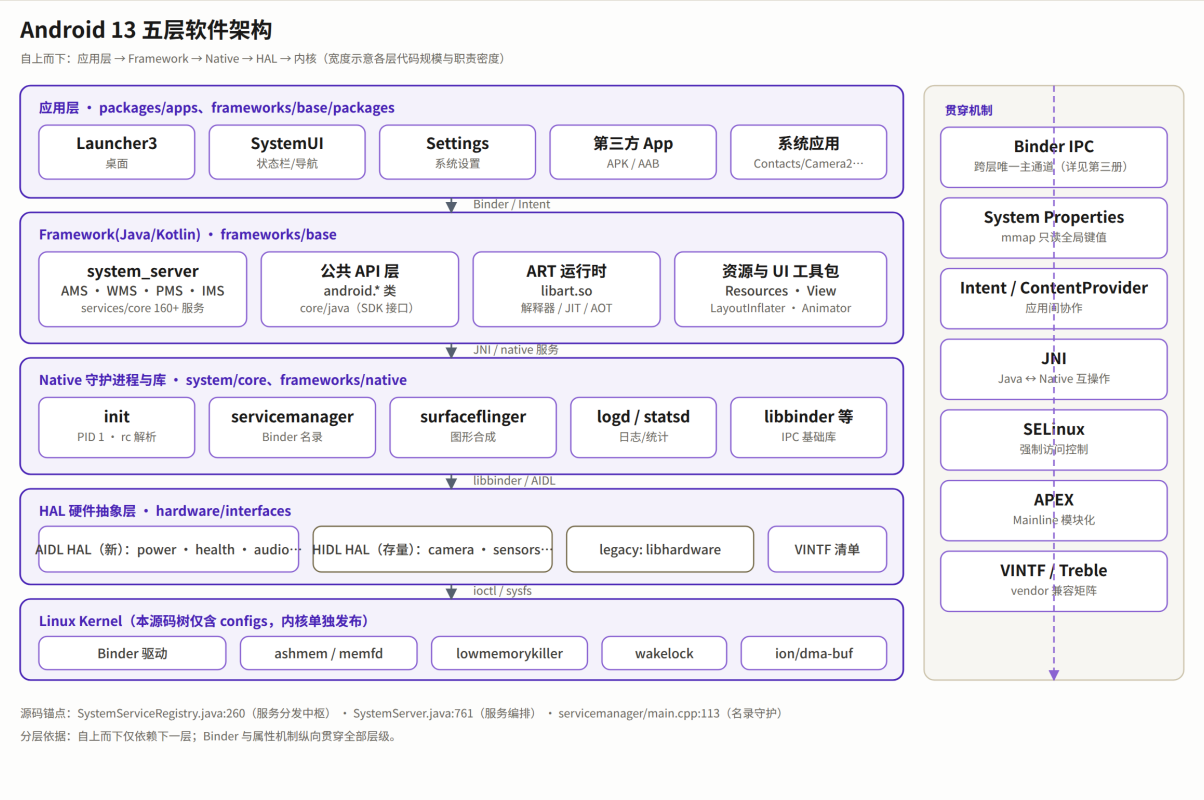


图 9 九跳纵贯五层架构：跳 1-2 在内核层、跳 3-6 跨 Framework 与 Native、跳 7-8 在应用层、跳 9 回到 Native 与硬件——一次点击对五层架构的完整采样

20.2 全链走读：一个 55 行的流程块

text 一次点击的全程（进程名缩写：SS=system_server，APP=应用进程，SF=surfaceflinger）

```
跳1-2 硬件与内核（本树外）
触摸 IC 扫描电容矩阵 → 触摸控制器固件合成坐标 → I2C/SPI 中断 → 内核 evdev 驱动
→ 事件进入 /dev/input/eventX 节点
// 讲解注释①：evdev 是内核对输入设备的统一抽象——上面的硬件千差万别，
// 下面的用户态只见 EV_ABS/EV_KEY/EV_SYN 三类码，硬件差异被驱动吃掉了
跳3 EventHub（SS 内，EventHub.cpp:1533 getEvents）
inotify 监控 /dev/input（:68 DEVICE_INPUT_PATH）增删设备；epoll 等待事件可读
→ 从 eventX read 出 RawEvent 数组返回
// 讲解注释②：EventHub 是"设备目录 + 事件泵"：设备能力（有哪些轴、多点几指）
// 经 EVIOCGBIT ioctl 查询，InputReader 据此选配置（单点屏/多点屏/触摸板）
跳4 InputReader（SS 内，InputReader.cpp:81 loopOnce）
mEventHub->getEvents() → processEventsLocked(:145) 按设备类型分派
→ CursorInputMapper/TouchInputMapper 累积采样、滤抖动、换算坐标系
→ mQueuedListener.flush() 把 NotifyMotionArgs 交给下一个环节
// 讲解注释③：Reader 线程是"从不阻塞"的专职采样员——它的唯一纪律是
// 尽快回到 getEvents，保证不丢硬件事件；加工规则（去抖/校准）全在这一跳
跳5 InputDispatcher（SS 内，InputDispatcher.cpp:3975 notifyMotion）
事件入队 → dispatchOnce(:602) → dispatchOnceInnerLocked(:718)
→ findTouchedWindowTargetsLocked(:1990)：谁是触摸点下的窗口?
// 讲解注释④：定窗依据来自 WMS 的窗口层级（:4645 setInputWindows 持续同步）——
// 这就是 InputManagerService 与 WindowManagerService 在 SystemServer:1570/:1586
// 相邻启动的原因：输入天然依赖窗口信息
跳6 跨进程注入（SS 到 APP，InputTransport.cpp:334 openInputChannelPair）
每个有焦点窗口的 App 在注册时与 SS 建立一对 socket（InputChannel）
```

```
→ startDispatchCycleLocked(:3214) 把 MotionEvent 写进 socket
→ dispatchEntry 同时进入 waitQueue——ANR 计时从这里开始（书5 详述）
// 讲解注释⑤：为什么是 socket 不是 Binder？①事件是高频单向流，
// 无需返回值的请求-响应开销；②dispatcher 需要对不响应窗口做超时裁决，
// socket 的 fd 可被 poll 统一等待；③fd 经 Binder 传递（书3 的能力传递）
跳7 App 消费（APP 主线程，ViewRootImpl.java:8819 deliverInputEvent）
WindowInputEventReceiver.onInputEvent → 输入管线 InputStage 逐站过滤
→ ViewPostImeInputStage.processPointerEvent(:6542)
→ mView.dispatchPointerEvent(event)——事件沿 View 树命中测试下传
// 讲解注释⑥：管线的存在是为了把 IME/无障碍/Preprocess 等"前置关卡"
// 与业务分发解耦——每一站决定 FINISH_HANDLED 或 FORWARD
跳8 渲染节拍（APP，Choreographer.java:742 doFrame）
点击引起的状态变化调用 invalidate → ViewRootImpl.scheduleTraversals(:2262 挂
CALLBACK_TRAVERSAL)
→ Choreographer.doFrame 按 INPUT/ANIMATION/TRAVERSAL/COMMIT 顺序回调
→ performTraversals(:2685) 测量布局绘制 → mThreadedRenderer.draw(:4579)
→ HardwareRenderer.syncAndDrawFrame(:456) → RenderProxy::syncAndDrawFrame
（libs/hwui/renderthread/RenderProxy.cpp:140）——RenderThread 接管，产出 GraphicBuffer 入队
BufferQueue
// 讲解注释⑦：主线程只"提交命令"，真正画在 RenderThread——
// 主线程卡住不阻塞渲染，这是书8 的核心议题；缓冲经共享内存传递（零拷贝大块数据）
跳9 合成送显（SF，SurfaceFlinger.cpp:1992 commit / :2164 composite）
VSync 到来 → commit：处理事务、latch 各层最新缓冲
→ composite：按 Z 序对每块屏幕构建合成参数 → 可硬件合成交 HWComposer，
不可合成的用 GPU 合成 → presentAndGetReleaseFences(HWComposer.cpp:508) 送显
→ 屏幕控制器在本帧扫出像素——按钮高亮出现在你的眼前
// 讲解注释⑧：SF 与 App 之间是生产者-消费者（BufferQueue，第十二章的"货币"）；
// 合成结果再经 DRM/KMS 送显（内核层）——控制流 Binder、数据流共享内存、
// 显示流帧栅栏同步，三条通道各司其职
```

20.3 逐跳精读：三个值得停下来想的设计

为什么 Reader 与 Dispatcher 要拆成两个线程？（InputManager.cpp:73 start 先起 Dispatcher 再起 Reader）采样与决策的节拍不同：硬件事件以微秒级到达，而"哪个窗口接收"可能因窗口切换、IME 弹起而复杂甚至需要等待。拆开后，Reader 永不阻塞地吃硬件事件，Dispatcher 可以从容等待窗口答案、批量合并移动事件、对无响应窗口起 ANR 计时。**合并决策集中在一处，是延迟与正确性的平衡点**。

为什么事件通道、渲染缓冲、服务调用用三种不同通道？回到第六章的机制选型表：事件是高频小数据单向流（socket）；图形缓冲是大数据（共享内存）；App 与系统的控制面是低频双向调用（Binder）。一次点击全链把三种通道各用了一遍——**"通道选型跟着数据形态走"不是教条，是这条链上每跳的设计依据**。

ANR 的种子在哪一跳埋下？跳 6：dispatchEntry 进入 waitQueue 的那一刻开始计时，超时阈值按窗口类型（前台/可见）不同。跳 7 的 App 主线程若被业务卡住，Dispatcher 在 SS 侧等待——**ANR 是"跨进程的对话超时"，不是 App 自己的异常**。第 21 章将补上最后一块拼图：被冻结（cgroup.freeze）的进程同样收不到调度，事件超时同样是必然结局。

设计洞察 · 逐段精读：为什么这条链"看起来绕"

有读者会问：点击为什么不直接进 App？绕 SS 一圈贵在三点：①**仲裁集中**——焦点、优先级、注入安全全由 SS 裁决，App 无法伪造触摸（信任锚在系统侧）；②**解耦**——App 崩溃不影响其他窗口的事件流，Dispatcher 会把死窗口从目标表摘除；③**可观测**——SS 侧集中埋点，全机输入延迟（LatencyAggregator）可度量。分散的直达路径快 1 微秒，集中的路径稳 10 年——系统设计的一贯取舍。

跨册交叉引用：书5 将逐函数展开跳 3-7（EventHub 设备扫描、Dispatcher 定窗算法、ANR 状态机、Choreographer 节拍）；书8 展开跳 8-9（RenderThread 命令流、BufferQueue 三缓冲、HWComposer 客户端/服务端合成决策）；书14 覆盖跳 1-2 的触摸驱动与设备树；书4 讲调度延迟对跳 7 的影响。本章排障落点见排障手册"场景十一：界面卡顿"（跳 8-9 延迟）与"场景七：进程被冻结"（跳 6 事件无人接收）。

第二十一章 进程优先级总览：从 oom_adj 到 cgroup 冻结

设计洞察 · 学习目标

1. 背出 adj 阶梯的关键档位与真实常量值；2. 画出 adj 从计算到消费的跨进程链路；3. 解释 lmkd 的 PSI 事件驱动模型；4. 分清冻结/查杀/内存压缩三件套的分工与触发条件

"为什么我的后台进程被杀了？""为什么切回 App 要重新冷启动？""为什么推送收不到？"——这三个问题共享同一个答案骨架：**进程优先级（oom_adj）**。书4 会逐函数拆 OomAdjuster 的打分规则，书9 会讲后台调度与电池；本章给整册读者一张"一页图解"：adj 这个数字是什么、谁计算、写给谁、内核与用户态各自怎么消费它。它是第二十一章也是全书"运行中的 Android"视角的收官——**第五章的静态服务表，加上本章的动态优先级，才是 system 的完整运行画像**。

21.1 adj 阶梯：一张必须刻在脑子里的表

adj（oom_score_adj）是内核认可的全机统一优先级标尺：**越大越先被杀，负数不可杀**。以下常量全部实测自 `frameworks/base/services/core/java/com/android/server/am/ProcessList.java:188-275`：

adj 值	常量	含义（ProcessList.java 行号）
-1000	NATIVE_ADJ	native 守护进程（:275）
-900	SYSTEM_ADJ	system_server 本尊（:271）
-800	PERSISTENT_PROC_ADJ	persistent 系统进程（:268）
-700	PERSISTENT_SERVICE_ADJ	常驻服务所在进程（:264）

adj 值	常量	含义（ProcessList.java 行号）
0	FOREGROUND_APP_ADJ	前台交互进程（:260）
100	VISIBLE_APP_ADJ	可见但非交互（:250）
200	PERCEPTIBLE_APP_ADJ	可感知（如后台放音乐，:246）
300	BACKUP_APP_ADJ	正在备份（:232）
500	SERVICE_ADJ	后台服务进程（:223）
600	HOME_APP_ADJ	桌面（:219）
700	PREVIOUS_APP_ADJ	上一个使用过的 App（:214）
900-999	CACHED_APP_MIN/MAX_ADJ	缓存进程区（:193/:192）
1001	UNKNOWN_ADJ	尚未定级的进程（:188）

注意三个"反直觉"排序：**桌面（600）排在普通服务（500）之后**——桌面必须常在，但桌面自己不会消耗成百个；**缓存区（900-999）是一个区间而非单值**——同区内部按 LRU 再排序，先杀最老的；**UNKNOWN_ADJ（1001）比 CACHED_APP_MAX_ADJ（999）还大**——未定级进程是最高危的，OomAdjuster 的职责就是尽快把它定级。

21.2 全链走读：adj 的一生（计算、下发、消费、冻结）

text adj 的一生：跨四个进程/层的完整链路（一条数字的旅行）

```
① 计算：OomAdjuster（services/core/java/com/android/server/am/OomAdjuster.java）
:123 class OomAdjuster；:428 updateOomAdjLSP 一次重算遍历全进程
// 讲解注释①：每个进程的最终 adj = 各维度候选值取 max（组件状态、窗口可见性、
// 服务重要性……书4 逐项展开）；触发时机是状态变化（onResume/服务绑定/广播派发）
② 下发：ProcessList.setOomAdj（ProcessList.java:1390-1414）
buf 依次写入 LMK_PROCPRIO(=1)+pid+uid+amt（:1399-1403，常量定义 :342）
→ writeLmkd 写 /dev/socket/lmkd
// 讲解注释②：下发不走 /proc 直写也不走 Binder，走 Unix socket 到 lmkd——
// 见 21.3 的设计洞察；耗时超 250ms 会打 "SLOW OOM ADJ" 警告（:1406-1408）
标准调用点：OomAdjuster.java:2587 ProcessList.setOomAdj(pid, uid, curAdj)
③ 消费一（查杀）：lmkd（system/memory/lmkd/lmkd.cpp）
PSI 内存压力事件驱动 mp_event_psi(:2544)——不是轮询！
→ 按 min_score_adj 从高到低找目标 find/kill_one_process(:2276)
→ 写 /proc/pid/oom_score_adj 查证并执行（写点 :1139）
→ 杀完打日志 "Kill 'xxx' (pid), uid, oom_score_adj N to free ..."（:2345/:2352）
// 讲解注释③：PSI 阈值（:221-222，1 秒内 some 停顿 70/100ms）与 ro.lmk.* 属性
//  （:121-158，device_config 可覆盖）共同决定"什么时候杀、杀到哪一档"
④ 消费二（冻结）：CachedAppOptimizer（services/core/java/com/android/server/am/CachedAppOptimizer.java）
:1649 freezeProcess——仅当 curAdj 大于等于 CACHED_APP_MIN_ADJ(900) 才继续
→ 第一步 freezeBinder(pid, true)（JNI com_android_server_am_CachedAppOptimizer.cpp:475-480
调 IPCThreadState::freeze）：先冻结 Binder 事务队列，清空在途事务防丢
→ 第二步 Process.setProcessFrozen(pid, uid, true)（android/os/Process.java:1106）
```

```
→ JNI android_util_Process.cpp:282 → SetProcessProfiles(uid, pid, {"Frozen"})
→ libprocessgroup 按 task_profiles.json:97 的 Frozen profile
写 freezer 控制器的 cgroup.freeze 文件（:76-78 定义 FreezerState 的落点）
// 讲解注释④：内核 cgroup v2 冻结是"整进程暂停调度"——信号、线程、定时器全部停摆，
// 比 STOP 信号更彻底且对进程透明；总开关在 Settings.Global.CACHED_APPS_FREEZER_ENABLED
//  （CachedAppOptimizer.java:136-137、:808-809 读取）
⑤ 消费三（压缩）：进程内存压缩（compaction）
setAdj 大于等于 CACHED_APP_MIN_ADJ 走一轮 reclaim/compact（ProcessList.java:1007-1009）
// 讲解注释⑤：冻结是"暂停"，压缩是"腾内存"（把冷页换出）——
// 三件套由同一个 adj 信号驱动，只是消费端不同
```

21.3 逐段精读：三个设计决策

setOomAdj 为什么走 socket 而不是 Binder 或直写 /proc？其一，**写点唯一**：oom_score_adj 只该由一个仲裁者写，lmkd 是唯一写点，它同时把 adj 记进自己的进程账本（红黑树），杀时直接按账本选目标，不必再扫 /proc。其二，**去耦内核接口**：内核 LMK 时代 AMS 直接写 /proc/.../min_free 与 oom_adj；PSI 时代接口大改——AMS 只面对一个稳定的 socket 协议（LMK_PROCPRIO 等命令族，:332 起注释），内核怎么演进与 Framework 无关。其三，**时序安全**：adj 下发发生在AMS 高峰期（启动、切前台），socket 单写者模型最不容易死锁。`SLOW OOM ADJ` 警告（:1406-1408）就是这条链的健康探针。

查杀为什么从内核搬回用户态（lmkd）？内核 LMK 只能看内存水位，看不懂"这个进程正在放音乐""那个进程 10 分钟前才被用户用过"——语义判断必须回到有完整信息的用户态。PSI（内核只提供压力信号）+ lmkd（用户态决策）的分工，是"**内核提供机制，用户态制定策略**"的教科书案例。代价是用户态响应比内核慢——所以还有 cgroup 冻结在前面挡住大多数内存恶化。

冻结与杀死是替代关系吗？不是，是**时间轴上的接力**：进程滑入缓存区（adj≥900）→ 冻结（立即，省 CPU/调度）；内存压力仍上升 → PSI 事件 → 按账本从高 adj 向低 adj 查杀（减法）；整过程中低优先级进程还会被压缩（整理）。冻结让"被杀"从常态变成最后手段——**这就是 Android 11 以后"后台 App 大概率没被杀、只是被冻结"的系统级解释**，也是书9 后台策略的物理基础。

> **设计洞察 · 设计洞察：adj 是一条"全机共识的接口"**
>
> 上面每一环——AMS 的组件状态、lmkd 的 PSI 策略、cgroup 的冻结开关、内核的 oom_score_adj 文件——都围绕一个 int 值协作，没有任何两环直接依赖彼此的内部结构。跨团队跨层共享一个**带明确语义的标量**，比共享复杂对象稳定得多；这是"用协议代替集成"的架构直觉，也解释了为什么书4/书9 讲的都是"adj 怎么算"，而消费端几十年几乎没变。

跨册交叉引用：书4 将逐函数拆解 OomAdjuster 的每条打分规则与 adj 回写时机；书9 承接缓存区管理、Doze 与桶对 adj 的修正；书8 关联渲染进程的优先级保障；书3 承接 IPCThreadState::freeze 的 Binder 冻结语义（冻结进程的在途事务如何排空）；第十一章的 SELinux 则守护这条链的起点（只有 system 域可写 lmkd socket）。本章排障落点见排障手册"场景六：进程反复被杀"与"场景七：进程被冻结后无响应"。

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. Android 软件栈分为哪五层？每层只依赖哪一层？

2. "贯穿列"的三条机制是什么？各自解决什么问题？

3. 为什么本源码树的 kernel/ 目录只有三个子目录？AOSP 的内核源码在哪？

4. HAL 从 legacy 演进到 HIDL 的动机是什么？HIDL 到 AIDL 又解决了什么？

5. system_server 中为什么不把 SurfaceFlinger 放进来？

6. Boot.img 和 init_boot 分区各自装什么？GKI 改变了什么？

7. ART 的三级执行策略是什么？谁决定一个方法用哪级？

8. APEX 模块化解决了什么问题？哪些模块适合 APEX 化？

9. 属性系统的"读快写慢"是什么意思？为什么这么设计？

10. Soong 为什么取代了 Make？两者的根本区别是什么？

第二级：分析题（检验机制理解）

11. **追踪调用链**：画出 context.getSystemService(Context.WIFI_SERVICE) 从 App 进程到 system_server 的完整路径，标出每次进程/线程切换。

12. **架构推演**：如果 Google 决定把 SystemUI 从独立进程移到 system_server 内，会带来什么问题？至少给出三条。

13. **对比分析**：Binder 与 Unix socket 都是 IPC，为什么 Android 选了 Binder？从安全、性能、能力传递三个维度对比。

14. **机制拆解**：preload() 加载了哪些内容到 Zygote？fork 后子进程如何继承？写时复制在这里起了什么作用？

15. **失败场景**：如果 boot.img 的 ramdisk 损坏，启动会在哪一步失败？系统会怎样？

16. **安全推演**：一个恶意 App 想伪造 getCallingUid() 返回 system uid（1000），在当前 Binder 架构下可能吗？为什么？

17. **版本演进**：Android 9 的 ClientTransaction 改造解决了什么问题？改造前后有什么区别？

第三级：设计题（检验系统思维）

18. **设计新机制**：假设你要设计一个"系统级配置变更通知"机制（比属性更实时、比 Binder 更轻量），你会怎么设计？考虑推/拉模型、安全、性能、向后兼容。

19. **架构决策**：你要给一个新 SoC 平台移植 Android，列出前十个必须实现的 HAL 及优先级。

20. **排障设计**：用户报告"开机后 30 秒才出现桌面"，给出分层排查方案（从内核到 Framework），每层至少一个检查命令。

参考答案要点

1. 应用层-Framework-Native-HAL-内核；每层只依赖正下方一层。

2. Binder（一切跨进程方法调用）、System Properties（全机键值状态广播）、SELinux（从 init 第一个动作到每次访问的强制访问控制）；Intent/ContentProvider 与 JNI 是前两者之上的应用协作与语言边界封装（第六章表格）。

3. 本树 kernel/ 只有 configs/prebuilts/tests；AOSP 内核是独立仓库（android13-5.10 等）；稳定 UAPI 头经 external/kernel-headers/original/uapi 进入编译；GKI 之下内核镜像由统一通道供应，与 system/vendor 三方解耦。

4. legacy→HIDL：进程隔离+接口版本化，vendor 代码不再编进 Framework 进程，system 可独立升级（Treble）；HIDL→AIDL：一套 IDL 编译器、一个 binder 域、一个服务管理器，工具链归一消灭双栈维护（第三章 3.2）。

5. 崩溃隔离+渲染实时性+内存配额独立（SF 的 SCHED_FIFO 线程不该跟 AMS 抢核）。

6. boot.img=内核+ramdisk；Android 13 新增 init_boot 承载通用 ramdisk，boot 只留内核；GKI 的改变是内核本身也进入"通用镜像+厂商模块"模式——内核、system、vendor 三方各自独立升级（第七章 7.1）。

7. 解释器起步、JIT 运行期兜底、AOT 按 profile 提前编译热点；决定者：安装期/空闲期 dex2oat 读 device/cloud profile 决定编译哪些方法，其余方法解释执行、运行时热点再被 JIT 捕获（第八章 8.1）。

8. 解决"必须随系统走、又想独立升级"的模块（ART、bionic、网络栈等）的发版节奏问题：apexd 运行时挂载、staged 会话支持试运行与自动回滚；适合"策略与 UI"类，不适合"事实与账本"类（第十六章 16.3 口诀）。

9. 读路径 mmap 共享区零系统调用（纳秒级）；写路径经 socket 到 init 单点仲裁（含 SELinux 检查与持久化）。设计原因：属性读写发生在系统最早期——连 Binder 与完整 SELinux 策略都未就绪，机制必须"笨"到不依赖任何运行时（第六章、第十章 10.3）。

10. Make/Kati 是全局变量+隐式规则的命令式体系，规模膨胀后解析慢、依赖不透明；Soong 用声明式 Android.bp+Go 实现+直接产出 ninja：增量快、依赖显式、错误在配置期暴露（第七章 7.2）。

11. App ContextImpl->SystemServiceRegistry Fetcher->ServiceManager.getService("wifi")->Binder十跳（书3）->system_server WifiService->BinderProxy->WifiManager->缓存。

12. ①故障域合并——SystemUI 的 ANR/崩溃直接拖垮整个 system_server（Watchdog 处决）；②实时性互扰——渲染线程与 160+ 服务、Binder 线程池抢核；③发布节奏绑死整机 OTA，失去 RRO/独立进程的灵活定制；④攻击面扩大——system uid 下任何漏洞直通系统核心。

13. 安全：Binder驱动采集UID/PID不可伪造 vs socket自报；性能：一次拷贝 vs 两次；能力：flat_binder_object传引用 vs 只传数据。

14. preload() 预加载资源与预加载类清单（ZygoteInit，:247 起）及常用 JNI 库；fork 出的子进程对这些只读页写时复制——加载一次、千百个进程共享，修改者才付复制成本（第八章）。

15. 第一阶段init无法挂载system分区->switch_root失败->内核panic或bootloop。

16. 不可能。getCallingUid 的值来自 Binder 驱动在事务里写入的发送方 euid（tr.sender_euid），由内核采集、用户态无法篡改；servicemanager 等仲裁者还能用 socket 的 SO_PEERCRED 交叉验证（第十章与第三册对照）。

17. 把生命周期调度抽象为 ClientTransaction（含各类 ActivityClientRecord 操作）一次 Binder 消息发给 App，由 App 侧 TransactionExecutor 统一执行——减少此前 AMS 与 App 之间多次细粒度往返，事务可合并、可编排（书4 展开调度语义）。

18. 评分点：读路径无锁 mmap 快照+版本号，写路径单点仲裁；推（通知 socket/fd 唤醒）拉（全量拉取回退）结合保证旧客户端兼容；前缀化权限与 SELinux 上下文；可持久化与可审计。对照第六章属性系统与 9 题——好答案应复用其不对称设计。

19. 前三决定能不能开机：Power>Health>Graphics(composer)>Audio>Camera>Sensors>Boot>Keymaster>Gatekeeper>DRM。

20. 分层方案：内核层（dmesg 查 panic/驱动超时）→ init 层（dmesg 看 init 两阶段、getprop init.svc.* 看服务状态机）→ system_server 层（logcat -b events 的 boot_progress 差值定段、Watchdog 日志）→ 服务级（perfetto traceBegin 段长、bootchart 甘特图）→ 应用层（Launcher 首帧、am stack/dumpsys activity）——与排障手册场景一/八的命令一一对应。

术语表（按拼音排序）

术语	全称	所在册	一句话解释
APEX	Android Pony EXpress	册1	可独立升级的模块容器（.apex），运行时挂载到 /apex
ART	Android Runtime	册1	Java 运行时：解释/JIT/AOT 三级执行
AVB	Android Verified Boot	册1/2	镜像完整性校验链（vbmetro->boot/system/vendor）

术语	全称	所在册	一句话解释
Binder	--	册3	核心 IPC：一次拷贝+内核身份+能力传递
BootConfig	Boot Configuration	册2	kernel cmdline 的结构化继任者
BSP	Board Support Package	册14	板级支持包（内核+设备树+HAL+sepolicy+固件）
COW	Copy-On-Write	册1/2	fork 后共享物理页，写时才复制
CTS	Compatibility Test Suite	册18	兼容性测试，通过=Google Play 预装权
DE/CE	Device/Credential Encrypted	册11	FBE 双域：开机即解 vs 解锁后解
dm-verity	Device Mapper Verity	册1/6	运行时只读分区块级校验
FBE	File-Based Encryption	册11	文件级加密（按目录分域）
GKI	Generic Kernel Image	册2	通用内核镜像：boot 只装 Google 签名内核
HAL	Hardware Abstraction Layer	册1/14	硬件抽象层（legacy->HIDL->AIDL 三代）
HIDL	HAL Interface Definition Language	册1/14	Treble 时代的 HAL IDL（已冻结）
init.rc	Android Init Language	册2	init 配置文件语法（service/on/import）
JIT	Just-In-Time	册1	运行时热点编译（ART 三级之一）
KeyMint	--	册11	硬件密钥管理 HAL（TEE/StrongBox）
Mainline	Android Mainline	册1	Google Play 系统更新机制（APEX 模块化）
MTE	Memory Tagging Extension	册2/4	ARMv8.5 硬件内存标签
NMS	NotificationManagerService	册13	通知管理（排序/通道/速率）
PMS	PackageManagerService	册6	包管理（扫描/安装/权限）
RIL	Radio Interface Layer	册19	电话框架与 modem 的抽象层（socket）
RRO	Runtime Resource Overlay	册1	运行时资源覆盖
SoC	System on Chip	册1/14	片上系统（CPU+GPU+ISP+modem）
Soong	--	册15	Android 构建系统（Android.bp->ninja）
TF	Tradefed	册18	测试调度框架
Treble	Project Treble	册1	system/vendor 架构解耦工程

术语	全称	所在册	一句话解释
UAPI	User API	册1/3	内核对用户空间的稳定接口
VHAL	Vehicle HAL	册16	车机 HAL（车速/档位等属性模型）
VINTF	Vendor Interface	册1/14	设备接口清单与兼容矩阵
VTS	Vendor Test Suite	册18	HAL 接口验证套件
WMS	WindowManagerService	册5	窗口管理（层级/焦点/Insets）
Zygote	--	册2	应用进程之母（预加载+COW fork）

排障手册（按"我的问题是..."索引）

使用方法：先按症状对号入座，再按**四件套**走完一次定位——**症状**（logcat/dmesg 关键字样例）→ **定位命令**（可直接在 adb shell 执行，默认 userdebug/eng 版）→ **根因**（括号内标注本册对应章节）→ **修复**。每一场景的根因都能在正文里找到源码锚点，手册是"把全书知识变成肌肉记忆"的最后一站。

场景一：开机卡在 logo / 开机动画无限转圈

症状：动画播超过 2 分钟不退；`getprop sys.boot_completed` 长时间为空。关键字样例：

```text
init: critical process 'zygote' exited 4 times in 4 minutes
// 出处 init service.cpp:335——zygote 四次重启仍失败，init 自身 FATAL，整机 bootloop
Watchdog: *** WATCHDOG KILLING SYSTEM PROCESS: Blocked in monitor ...
// 出处 Watchdog.java:840——system_server 关键线程超时被看门狗处决
```

定位命令：

```bash
adb shell getprop sys.boot_completed # 空即未完成；唯一置位点在 AMS.java:5149
adb shell dmesg | grep -iE "critical|watchdog|panic"
adb logcat -b events -d | grep boot_progress
adb logcat -d | grep -E "SystemServer|Zygote |Watchdog"
adb shell dumpsys -l >/dev/null && echo alive || echo system_server_dead
```

根因：三类——startOtherServices 某服务构造/onStart 抛异常被包成 RuntimeException 拉崩 system_server（第十九章 19.2 的"自杀式容错"）；关键线程阻塞触发 Watchdog（:310/:312 的 "Blocked in monitor/handler" 给出线程名）；更早阶段被 SELinux 或内核 panic 挡住。

修复：取 logcat 中最后一个 traceBegin 段名定位到具体服务（与 perfetto 段同名）；watchdog 场景按 Blocked 线程回查它等待的锁；对比 boot_progress 时戳找"最后一个成功里程碑"向后二分。深入：书2（启动主线）+ 书9（后台调度）。

场景二：服务找不到（getService 返回 null / 调用崩溃）

症状：

```text
java.util.NoSuchElementException: No service published for: xxx
// 出处 ServiceManager.java:365 的 ServiceNotFoundException——getService 查无此名
```

也可能第一次调用卡数秒才成功（那是 lazy 服务被 tryStartService 拉起，第十九章 19.5）。

定位命令：

```bash
adb shell service list | grep xxx # servicemanager 名录（过 canList 的 SELinux 检查）
adb shell getprop init.svc.xxx # init 视角：running/restarting/stopped
adb shell dumpsys -l | grep xxx # dumpsys 注册与 Binder 名是两本账，两边都查
adb logcat -d | grep -iE "avc.*xxx|denied"
adb logcat -d | grep "Starting .*xxx" # SSM 注册日志（SystemServiceManager.java:210）
```

根因：三种——服务进程没起来（init.svc 非 running）；addService 被 servicemanager 七道闸之一拒绝（App UID/SELinux denial/VINTF 未声明，第十九章 19.4 表）；客户端查早了（启动顺序依赖，服务尚未发布）。

修复：进程起不来先看 avc 与 init.svc 状态机；SELinux 补 service_contexts 与 allow 域 ... service_manager find；VINTF 补 vintf_fragments；查早了改用 onBootPhase 或 waitForService。深入：书3（Binder）+ 书14（VINTF 检查）。

场景三：跨进程调用报 TransactionTooLargeException（服务调用版"413"）

症状：

```text
E/JavaBinder: !!! FAILED BINDER TRANSACTION !!! (parcel size = 1234567)
// 出处 android_util_Binder.cpp:875——随后可能抛 TransactionTooLargeException
```

定位命令：

```bash
adb logcat -d | grep "FAILED BINDER TRANSACTION" # 抓 parcel size 与调用点
adb logcat -d | grep -B5 "TransactionTooLargeException"
```

根因：Binder 事务缓冲区每进程约 1MB，且由**全部并发事务共享**——不是"单条不能超 1MB"，而是高峰期总量够不够。等价于 HTTP 413 Payload Too Large 在 Binder 世界的投影。

修复：大数据（Bitmap/大列表/文件内容）改走 ashmem/memfd 或 ContentProvider 流式；拆分事务；失败后等内存水位回退再重试，不要立刻硬顶。深入：书3（Binder 缓冲区管理）。

场景四：SELinux denied 三板斧

症状：

```text
avc: denied { find } for service=xxx pid=4321 scontext=u:r:myapp:s0
tcontext=u:object_r:activity_ser
vice:s0 tclass=service_manager
```

定位命令：

```bash
adb shell getenforce # 确认 enforcing（Permissive 说明已被临时放开）
adb logcat -b all -d | grep avc # main/events/kernel 三块都要收
audit2allow -i avc.log # 生成候选 allow（仅参考，绝不可照抄）
grep -rn "目标类型名" system/sepolicy/ # 回源码树反查类型定义与现有 allow
```

根因：策略缺最小 allow。三板斧：①翻译日志——denied 后大括号是权限、scontext 是发起域、tcontext 是目标类型、tclass 是类别（第十一章 11.2 的三张 contexts 表）；②判断该不该放——若 neverallow 已挡（编译期冲突，domain.te:119/:137 起大量条款），说明要改架构而不是补权限；③补最小 allow 并同步补 contexts。

修复：域缺失补 .te（第十六章 16.1 四步动作）；标签缺失补 file_contexts/service_contexts/property_contexts；调试期 setenforce 0 只用于确认"就是它"，修复必须落回策略。深入：书16（策略实战）。

场景五：属性写不进（setprop 无效）

症状：

```text
init: Unable to set property 'xxx' from uid:10123 gid:10123 pid:4321: SELinux permission
check faile
d
// 出处 property_service.cpp:578/:606 的拒绝日志
```

也可能无日志静默失败（返回码非 0）。

定位命令：

```bash
adb shell setprop debug.foo bar; echo $? # 非 0 即被拒
adb logcat -d | grep -i "Unable to set property"
adb shell getprop | grep debug.foo
# 回源码树查该前缀的 SELinux 上下文（对照第十章）
grep -rn "debug\.\|persist\." system/sepolicy/private/property_contexts
```

根因：四道闸之一（property_service.cpp:176 起）：名字/长度非法；ro. 前缀已设过（PREADONLY）；persist 未就绪；SELinux property_contexts 未授权（第十章 10.2）。

修复：选对前缀与写入时机；自定义前缀要在 property_contexts 定义类型并补 allow 域 ... property_service set；ro. 属性只能在构建期写入。深入：书2（init 仲裁细节）。

场景六：某进程反复被杀（lmkd 循环查杀）

症状：

```text
lmkd: Kill 'com.example.app' (12345), uid 10123, oom_score_adj 900 to free 350000kB rss
// 出处 lmkd.cpp:2345/:2352——logcat -s lmkd 的标准击杀记录
```

定位命令：

```bash
adb logcat -d | grep -E "lmkd|lowmemory"
adb shell dumpsys activity processes | grep -A4 com.example.app # setAdj/curAdj/状态
adb shell cat /proc/$(pidof com.example.app)/oom_score_adj
adb shell cat /proc/pressure/memory # PSI 实时压力
```

根因：进程处于缓存区（adj 900-999，第二十一章 21.1）且 PSI 越过阈值（lmkd.cpp:221-222 的 1 秒 70/100ms 停顿窗口；ro.lmk.* 可调，:121-158）。反复被杀两种病灶：泄漏持续推高全机压力；或进程身负重活却无前台态、adj 长期躺在 900。

修复：泄漏用 dumpsys meminfo 两次快照对比找增长点；重活进程声明前台组件或拆独立进程；设备侧调 ro.lmk.*（persist.device_config.lmkd_native.* 可覆盖）。深入：书4（进程优先级）+ 书9（后台）。

场景七：进程被冻结后无响应（没被杀但"死"了）

症状：后台 App 切回慢、推送回调迟到；dumpsys 显示 frozen=true；输入事件/网络回包无人消费（第二十章跳 6 的 socket 写端等不到读端）。

定位命令：

```bash
adb shell dumpsys activity processes | grep -i frozen
adb shell "find /sys/fs/cgroup -name cgroup.freeze 2>/dev/null | head"
# 对照实验：临时关闭冻结（设置键实测为 cached_apps_freezer，Settings.java:16542）
adb shell settings put global cached_apps_freezer false
```

根因：adj 大于等于 900 且满足条件时 CachedAppOptimizer.freezeProcess（:1649）两步走：先 freezeBinder（IPCThreadState::freeze）排空在途事务，再经 Frozen profile 写 cgroup.freeze（第二十一章 21.2 第④段）。冻结期间调度、定时器、信号全停。

修复：多为预期行为；若前台进程被误冻结，查 OomAdjuster 给它算出的 adj 是否偏低；冻结引发的 Binder 超时要连同客户端 timeout 一起看。深入：书4（冻结机制）+ 书9（冻结策略）。

场景八：开机慢

症状：sys.boot_completed 时戳晚于基线；某些 boot_progress_* 事件间隔异常大。

定位命令：

```bash
adb shell logcat -b events -d | grep boot_progress # 里程碑时戳差值即各段耗时
adb shell dmesg | grep init # init 两阶段与 class_start 日志
adb shell dumpsys activity | head -40 # 启动完成状态
```

根因：某服务 onStart 慢（perfetto 对应段长）；dexopt/首次编译占时；rc 触发链被阻塞；早期 HAL 未就绪拖住依赖链。

修复：perfetto 对照 boot_progress 定位最慢一段；bootchart 甘特图（4.3）看服务级耗时；单模块 m + adb sync 缩短迭代。深入：书2（启动主线）+ 书9（后台调度）。

场景九：没声音

症状：播放无声但有 UI；或仅某 App 无声。

定位命令：

```bash
dumpsys media.audio_flinger # Track 状态（started/muted/被移除）
dumpsys media.audio_policy # 路由与设备可用性
dumpsys audio # 音量与焦点归属
```

根因：音频焦点被抢或被静音、路由决策错误、audioserver 或音频 HAL 崩溃后的重启窗口（期间全静音）。

修复：先焦点与路由，再查 audioserver 日志与 HAL 存活（ps -A | grep audio）。深入：书12（音频管线）。

场景十：通知不显示

症状：notify() 调用成功但通知栏无显示。

定位命令：

```bash
dumpsys notification --noredact # 排名、拦截原因、挂起状态
dumpsys notification_policy # 勿扰与通道策略
```

根因：勿扰开启、channel 被用户关闭、PostNotification 运行时权限未授予、桶限制压制（NMS 的拦截决策都记录在 dumpsys 输出里）。

修复：按 dumpsys 给出的 ranking/intercept 原因逐项排除。深入：书13（通知管线）。

场景十一：界面卡顿

症状：滑动掉帧；perfetto FrameTimeline 出现红色帧。

定位命令：

```bash
dumpsys gfxinfo <包名> # 帧统计与最重帧
dumpsys SurfaceFlinger --latency # SF 侧合成时戳
perfetto # FrameTimeline 标注责任段
```

根因：主线程重活（第二十章跳 7）、RenderThread 过载（跳 8）、GC 停顿（第八章 CC）、SF 合成超节拍（跳 9）——FrameTimeline 直接标注责任方。

修复：按责任段归位：主线程任务异步化；大图降采样；分配热点查 GC 压力。深入：书5（Choreographer）+ 书8（渲染）。

场景十二：网络不通

症状：完全无网，或特定域名失败。

定位命令：

```bash
ping 基础连通
dumpsys connectivity # 网络评分与默认网络
dumpsys dnsresolver # DNS 配置与解析
dumpsys netd # netd 侧规则
```

根因：评分制下"连上"不等于"默认网络"（低分网不接管）；DNS 解析失败被误判为断网；VPN/always-on 规则拦截。

修复：按层级二分——链路（ping）、选路（connectivity）、解析（dnsresolver）、策略（netd）。深入：书10（网络栈）。

场景十三：编译太慢

症状：m 一跑几十分钟。

定位命令：

```bash
m nothing # 验证环境与语法（最快路径）
m <单模块名> # 单模块增量
adb root && adb sync # 增量部署替代整镜像
```

根因：全量构建与增量的差异被忽略；模块依赖图过大；soong 缓存冷。

修复：坚持单模块迭代（7.4 教程流）；不要 clean；用 adb sync 替代刷机验证。深入：书15（构建系统）。